RTL Design Sherpa

mem-ctrl-ip Common IP Micro-Architecture Specification 0.1

October 10, 2026

Table of Contents

1 Common Mas Index....10

2 mem-ctrl-ip Family — Overarching Hardware Architecture Specification....11

2.1 Document Organization....12

2.1.1 Front Matter....12

2.1.2 Chapter 1: Introduction....12

2.1.3 Chapter 2: The Plane Stack....12

2.1.4 Chapter 3: The DFI Generations....12

2.1.5 Chapter 4: Doctrines Added by the Common-IP Campaign....12

2.1.6 Chapter 5: The Line-Up....12

3 Document Information....13

3.1 Revision history....14

3.2 Chapter conventions....15

4 Chapter 1: mc_common_pkg — the family package....16

4.1 1.1 Package role....17

4.2 1.2 memtype_e....18

4.3 1.3 Shared type inventory....19

4.3.1 dram_op_e encoding....20

4.3.2 decoded_addr_t field layout....21

4.4 1.4 Helper-function implementation constraint....22

4.5 1.5 What stays rock-side....23

4.5.1 Legacy CSR widening argument....23

4.6 1.6 Migration plan reference....24

4.7 1.7 Open question: do the rocks import mc_common_pkg directly?.....25
5 mc_axi4_plane — AXI4 host interface and read-return ring.....26
5.1 Child-instance map.....27
5.2 Burst chop, split, and final coalescing.....30
5.3 Read return ring: ordering contract.....31
5.4 Parameters.....32
5.5 Ports.....34
5.5.1 Clock, reset, address map, and status.....34
5.5.2 Host AXI4 slave.....34
5.5.3 Downstream interfaces.....37
5.6 What this plane does not know.....41
5.7 Open questions.....42
5.8 References.....43
6 mc_storage_plane — Storage Plane and Entry-Pool FUBs.....44
6.1 mc_storage_plane parameters.....45
6.2 mc_storage_plane ports.....47
6.3 Internal wiring.....51
6.4 mc_wr_data_cam — write-data CAM / write buffer.....52
6.5 mc_rd_cmd_cam — read-command scheduling window.....56
6.6 mc_addr_mapper — flat address to DRAM tuple.....58
6.7 Open questions.....60
6.8 References.....61
7 Chapter 4 — Scheduler Plane and Pick Machinery.....62
7.1 1. Top-level parameters.....63

7.2 2. Top-level ports..... 66
7.3 3. Internal structure..... 72
7.4 4. Schedulable-entry view and unified prospect pool..... 74
7.5 5. Generic maintenance slot..... 75
7.6 6. Bank and global timer parameterization..... 76
7.7 7. Output command descriptor..... 77
7.8 8. Fire-stage authority..... 78
7.9 9. Page-policy context..... 80
7.10 10. P1 init accept/ack gating..... 81
7.11 11. Open questions..... 82
7.12 References..... 83
8 mc_training_plane — LPDDR2 training clients on the generic shell..... 84
8.1 Composition..... 85
8.1.1 Slot mapping..... 85
8.1.2 Maintenance-slot arbitration (shell)..... 86
8.1.3 One-active trn_cmd mux (shell)..... 86
8.1.4 mc_clk/dfi_clk CDC placement (shell)..... 86
8.1.5 Status aggregation..... 86
8.2 Boundary doctrine: training searches stay PHY-side..... 87
8.3 Parameters..... 88
8.4 Ports..... 89
8.4.1 Clock and reset..... 89
8.4.2 Runtime enable / build guard..... 89
8.4.3 CSR controls..... 89

8.4.4 Maintenance command channel to the scheduler....90
8.4.5 DFI-plane calibration sideband....91
8.4.6 Telemetry / status....92
8.5 Product integration....94
8.5.1 Pumice....94
8.6 ZQ placement as a maintenance client....95
8.7 Plane-only versus pumice-proven....96
8.8 Open questions....97
9 Chapter 6 — mc_ras_plane....98
9.1 Fixed decisions....99
9.2 Parameters....100
9.3 Ports....101
9.3.1 Write-data forward seam (upstream → downstream)....101
9.3.2 Read-return backward seam (downstream → upstream)....101
9.3.3 Reserved CSR surface....102
9.4 Bypass property....104
9.5 Explicitly not decided....105
9.6 Open questions....106
10 mc_dfi_2p1_plane — DFI 2.1 Command / Write / Read Pipeline....107
10.1 Purpose....108
10.2 Sub-FUBs and clock domains....109
10.3 The single controller↔PHY clock-domain crossing....110
10.4 DFI_RATE gearing and bus geometry....111
10.5 Command path....112

10.6 Write serializer....113
10.7 Read aligner....114
10.8 PHY timing knobs....115
10.9 Parameters....116
10.10 Ports....118
10.10.1 Controller domain (ctl_clk)....118
10.10.2 PHY DFI domain (dfi_clk)....118
10.11 Open questions....121
11 Chapter 8 — mc_dfi_3p1_plane....122
11.1 Purpose and scope....123
11.2 What is inherited unchanged from the 2.1 pipeline....124
11.3 What the 3.1 delta adds....125
11.3.1 Command path: mc_dfi_3p1_cmd_path....125
11.3.2 Write-leveling CDC....125
11.3.3 Calibration sideband removed....126
11.4 Parameters....127
11.5 Ports....130
11.5.1 Controller domain (ctl_clk)....130
11.5.2 PHY DFI domain (dfi_clk)....130
11.5.3 Write-leveling crossing ports....132
11.6 Family-boundary doctrine....134
11.7 Open questions and verification notes....135
12 mc_dfi_4p0_plane — DFI 4.0 Generation Delta....136
12.1 Inherited shared pipeline....137

12.2 What the 4.0 delta adds....138
12.2.1 Command path (mc_dfi_4p0_cmd_path)....138
12.3 What the plane leaves unimplemented....139
12.4 Parameters....140
12.4.1 mc_dfi_4p0_plane....140
12.5 Ports....142
12.5.1 mc_dfi_4p0_plane — controller domain (ctl_clk)....142
12.5.2 mc_dfi_4p0_plane — PHY DFI domain (dfi_clk)....143
12.5.3 mc_dfi_4p0_cmd_path — dfi_clk domain....144
12.6 Family boundary rule....146
12.7 Verification record....147
13 mc_init_plane — DRAM Initialization Plane....148
13.1 Responsibility....149
13.2 Parameters and ports....150
13.3 The two JEDEC sequences....152
13.4 Configuration surface....153
13.5 Verification....154

List of Figures

No figures in this document.

List of Tables

Table 1: Table 8.1: mc_dfi_3p1_plane parameters....126
Table 2: Table 8.2: Controller-domain ports....129
Table 3: Table 8.3: PHY DFI-domain ports....129
Table 4: Table 8.4: Write-leveling crossing ports....131

1 Common Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 mem-ctrl-ip Family — Overarching Hardware Architecture Specification

Version: 0.3 **Date:** 2026-10-20 **Status:** v0.1, the vision edition. This book is the family-level HAS: the plane-stack architecture the partner set out during the Phase-2 common-IP campaign, written down as the binding contract the rocks and the products are built against. The per-block microarchitecture lives in common_mas; this book owns the shape of the whole and the rules no single block owns.

> **Read this first.** A controller book (pumice/scoria/andesite HAS/MAS) and a family doc disagree → the family doc wins and the book is the defect. That rule predates this book (the family INDEX states it); this book is one of the docs it protects. Every claim here is either (a) measured in a named rock, (b) a cited JEDEC/DFI clause, or (c) an explicitly marked open question. Nothing else is asserted.
>
> **Warning:** every timing CSR in this family is a count of MC-clock cycles, correct only at the MC clock frequency it was derived for. A frequency change invalidates the whole timing set at once; software must re-derive and reprogram it before traffic resumes. Stale values present as data corruption, not errors. Each rock's HAS carries this warning at the register map; this book makes it family doctrine (Ch 4).

2.1 Document Organization

Note: All chapters linked below for automated document generation.

2.1.1 Front Matter

- Document Information

2.1.2 Chapter 1: Introduction

- Purpose, Scope, and the Two Lines

2.1.3 Chapter 2: The Plane Stack

- The Architecture: axi4 → storage → scheduler → (ras) → training → dfi
- The Descriptor Seam

2.1.4 Chapter 3: The DFI Generations

- Why `dfi_2p1_plane` / `dfi_3p1_plane` / `dfi_4p0_plane` Are Separate Blocks

2.1.5 Chapter 4: Doctrines Added by the Common-IP Campaign

- New Family Doctrines

2.1.6 Chapter 5: The Line-Up

- The common-ip Layout
- Products and Research: granite / gabbro / diorite

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Document Information

Field	Value
Document	mem-ctrl-ip Common IP — Micro-Architecture Specification
Version	0.3
Date	2026-10-20
Status	Partner-review draft, land-and-review with the common_has v0.1 book
Owner	mem-ctrl-ip family (common-ip/docs/)
Scope	One chapter per common block: package, eight planes, and the shared FUBs inside their owning plane’s chapter

3.1 Revision history

Version	Date	Change
0.1	2026-10-10	First edition, written against the Phase-2 extraction as landed: `mc_common_pkg`, eight planes (`mc_axi4_plane`, `mc_storage_plane`, `mc_scheduler_plane`, `mc_training_plane`, `mc_ras_plane`, `mc_dfi_2p1_plane`, `mc_dfi_3p1_plane`, `mc_dfi_4p0_plane`), and seventeen shared FUBs. Skeleton blocks (`mc_ras_plane`, `mc_training_plane` shell, per-generation DFI deltas) are documented honestly thin.
0.2	2026-10-20	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.3	2026-10-20	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

3.2 Chapter conventions

- RTL citations use `path:line` against `common-ip/rtl/` unless noted.
- Provenance citations use the rock books (`pumice_mas`, `scoria_mas`, `andesite_mas`) and the rock bug/issue records (pumice BUG-021, scoria BUG-001/002, ISSUE-002, and the campaign ledger).
- Parameter/knob detail lives in the knobs inventories (`mc_*_knobs.md` beside this book) and is summarized, not duplicated.
- The evidentiary rule (index “Read this first”) applies to every chapter.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Chapter 1: mc_common_pkg — the family package

mc_common_pkg is the single source of family-shared types for the memory-controller research IPs (pumice DDR2/LPDDR2, scoria DDR3/LPDDR3, andesite DDR4/LPDDR4). The package was extracted on 2026-10-10 during the Phase 2 common-IP reorganization (includes/mc_common_pkg.sv:17-18). Its encodings are the family encodings; the design authority is family doc 01 (01_mem_ctrl_pkg.md). The plane-stack context is in the family HAS (common_has_index.md); this chapter documents only the package contents and the rock-side boundary.

4.1 1.1 Package role

`mc_common_pkg` holds the types that were previously duplicated, near-identically, across `pumice_pkg`, `scoria_pkg`, and `andesite_pkg` (`includes/mc_common_pkg.sv:19-23`). It is imported directly by every common plane and shared FUB in `common-ip/rtl/` (for example, `macro/mc_axi4_plane.sv:178`, `macro/mc_scheduler_plane.sv:26`, `fub/mc_cmd_arbiter.sv:34`, `fub/mc_addr_mapper.sv:55`). The package contains no parameters, no build-time knobs, and no CSR definitions; it is a pure type-and-function package meant to be imported with `::*` at module scope.

4.2 1.2 memtype_e

memtype_e is a 3-bit packed enum (includes/mc_common_pkg.sv:41-48) whose encoding is owned by family doc 01 Table 1.0. Bit [2] is the LP axis; bits [1:0] form a generation field.

memtype_e	Member	memtype_e[1:0]	memtype_e[2] LP
3'b000	MEMTYPE_DDR2	2'b00 (generation 2)	0
3'b001	MEMTYPE_DDR3	2'b01 (generation 3)	0
3'b010	MEMTYPE_DDR4	2'b10 (generation 4)	0
3'b100	MEMTYPE_LPDDR2	2'b00	1
3'b101	MEMTYPE_LPDDR3	2'b01	1
3'b110	MEMTYPE_LPDDR4	2'b10	1
3'b011, 3'b111	reserved	2'b11 (generation 5, basalt)	—

The reserved codes (3'b011 and 3'b111) are illegal and must decode as an error rather than alias to a legal member (includes/mc_common_pkg.sv:36-39).

4.3 1.3 Shared type inventory

The package exports the following enums, struct, and helper functions. Every item is used by at least one common plane or FUB.

Symbol	Kind	File:line	Notes
`memtype_e`	enum	`includes/mc_common_pkg.sv:41-48`	3-bit, six legal members, two reserved codes
`dram_op_e`	enum	`includes/mc_common_pkg.sv:59-78`	5-bit superset of legacy 4-bit op tables; `OP_MPC = 5'h10` is the LPDDR4 addition
`bank_state_e`	enum	`includes/mc_common_pkg.sv:84-92`	bank-machine state
`page_policy_e`	enum	`includes/mc_common_pkg.sv:101-105`	open/close plus reserved; pumice's `PAGE_POLICY_HAPPY_HYBRID` (2'h2) was retired 2026-08-25
`decoded_addr_t`	packed struct	`includes/mc_common_pkg.sv:114-119`	rank/bank/row/col, padded to 4/4/18/14 bits
`is_column_op`	function	`includes/mc_common_pkg.sv:128-130`	true for RD/RDA/WR/WRA
`is_write_op`	function	`includes/mc_common_pkg.sv:132-134`	true for WR/WRA
`is_read_op`	function	`includes/mc_common_pkg.sv:136-138`	true for RD/RDA
`is_refresh_op`	function	`includes/mc_common_pkg.sv:140-142`	true for REF/REFPB
`has_auto_pre`	function	`includes/`	true for RDA/WRA

Symbol	Kind	File:line	Notes
		mc_common_pkg.sv :144-146	
is_zq_op	function	includes/ mc_common_pkg.sv :148-150	true for ZQCS/ZQCL

4.3.1 dram_op_e encoding

dram_op_e is widened to 5 bits so that OP_MPC exists once in the family (includes/mc_common_pkg.sv:21-23). Values 5'h00 through 5'h0F are identical to every rock's legacy 4-bit table (includes/mc_common_pkg.sv:56-57); the wire-level translation per memtype lives in each rock's DFI command formatter.

Value	Member	Meaning
5'h00	OP_NOP	no operation
5'h01	OP_ACT	activate
5'h02	OP_RD	read
5'h03	OP_RDA	read with auto-precharge
5'h04	OP_WR	write
5'h05	OP_WRA	write with auto-precharge
5'h06	OP_PRE	precharge
5'h07	OP_PREA	precharge all banks
5'h08	OP_REF	all-bank refresh
5'h09	OP_REFPB	per-bank refresh
5'h0A	OP_MRS	mode register set
5'h0B	OP_ZQCS	ZQ calibration short
5'h0C	OP_ZQCL	ZQ calibration long
5'h0D	OP_SREFE	self-refresh entry
5'h0E	OP_SREFX	self-refresh exit
5'h0F	OP_DPDE	deep-power-down entry
5'h10	OP_MPC	LPDDR4 multipurpose command; CA-bus encoding per the

Value	Member	Meaning
		andesite kmap book

4.3.2 `decoded_addr_t` field layout

Field	Width	Notes
`rank`	4 bits	pad to 4; actual width is `$clog2(NUM_RANKS)`
`bank`	4 bits	pad to 4; DDR3 uses 3
`row`	18 bits	pad to 18; MT41J256M16 uses 15
`col`	14 bits	pad to 14; MT41J256M16 uses 10

4.4 1.4 Helper-function implementation constraint

Each helper returns a single-line expression (`includes/mc_common_pkg.sv:124-126`). The RTL comment records that multi-line return expressions break yosys' frontend, which is why pumice's formal flow historically had to sv2v-flatten wrappers that imported its package.

4.5 1.5 What stays rock-side

The three rock packages still exist, but their contents are now thin.

Rock package	Remaining content	File
`scoria_pkg`	import/export shim for `mc_common_pkg` only	`research-ip/scoria-ddr3-lpddr3/rtl/includes/scoria_pkg.sv:29-33`
`andesite_pkg`	import/export shim for `mc_common_pkg` only	`research-ip/andesite-ddr4-lpddr4/rtl/includes/andesite_pkg.sv:30-34`
`pumice_pkg`	import/export shim plus `addr_map_scheme_e` and `odt_rule_e`	`research-ip/pumice-ddr2-lpddr2/rtl/includes/pumice_pkg.sv:28-60`

`addr_map_scheme_e` is retained only because retired macro/OLD modules still reference the type in regression sentinels (`pumice_pkg.sv:36-40`). `odt_rule_e` is pumice-specific ODT rule select (`pumice_pkg.sv:49-59`).

4.5.1 Legacy CSR widening argument

The rocks' `PHY_TIMING.memtype` CSR field stays 1-bit wide for Phase 2, with per-rock legacy values (`includes/mc_common_pkg.sv:25-27`; mc_common_pkg_knobs.md). Pumice maps 0→`MEMTYPE_DDR2` and 1→`MEMTYPE_LPDDR2` at `research-ip/pumice-ddr2-lpddr2/rtl/top/pumice_top.sv:199-207`. Scoria maps 0→`MEMTYPE_DDR3` and 1→`MEMTYPE_LPDDR3` at `research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_top.sv:237-245`. Andesite's CSR rebuild is a follow-up: `andesite_core.sv` currently takes `memtype_i` as a top-level port (`research-ip/andesite-ddr4-lpddr4/rtl/top/andesite_core.sv:90-91`) and no `andesite_top.sv` exists yet, so the legacy cast site is not yet in RTL.

The same config block in pumice and scoria also carries a page-policy cast warning: `REFRESH_TUNING.page_policy_or` uses a software encoding while `page_policy_e` uses OPEN=0/CLOSE=1. The RTL comment records that the old raw cast made software-OPEN run CLOSE and software-CLOSE run HYBRID, corrupting the open_page/reorder config axis (`research-ip/pumice-ddr2-lpddr2/rtl/top/pumice_top.sv:208-212`, `research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_top.sv:246-250`; issue #42). Note that `mc_common_pkg::page_policy_e` retired the HYBRID member in favor of `PAGE_POLICY_RSVD = 2'h3` (`includes/mc_common_pkg.sv:97-105`).

4.6 1.6 Migration plan reference

The migration that created `mc_common_pkg` is the Phase 2 migration deferred in family doc 01 (01_mem_ctrl_pkg.md). The conditions were: execute once, when andesite RTL bring-up starts, with both shipping controllers moving together. The knob inventory (mc_common_pkg_knobs.md) records what moved, what stayed, the legacy-CSR ruling, and the width-change consequences expected during adoption (e.g., `dram_op_e` re-width 4→5 and `memtype_e` re-width 1→3).

4.7 1.7 Open question: do the rocks import `mc_common_pkg` directly?

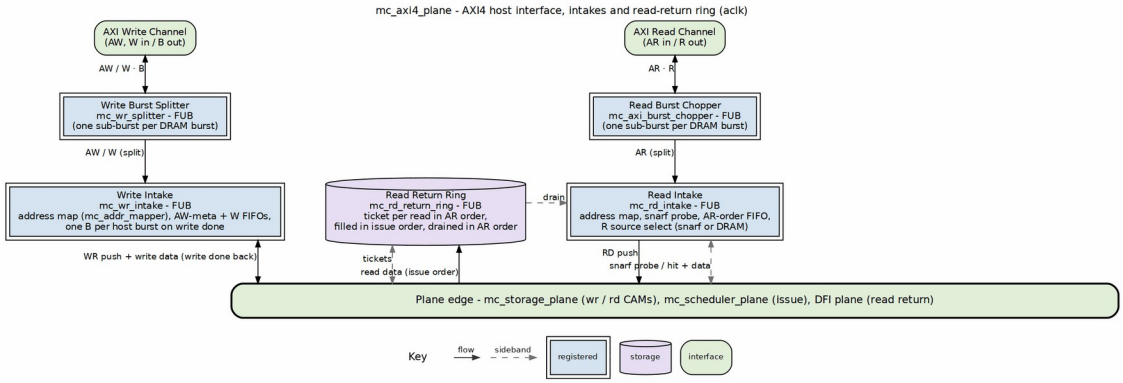
No. As of the Phase 2 extraction, common planes and FUBs import `mc_common_pkg::*` directly, but rock RTL still imports its own rock package (`pumice_pkg::*`, `scoria_pkg::*`, or `andesite_pkg::*`). Each rock package in turn `imports` and `exports` `mc_common_pkg::*` (`research-ip/scoria-ddr3-lpddr3/rtl/includes/scoria_pkg.sv:31-32`, `research-ip/andesite-ddr4-lpddr4/rtl/includes/andesite_pkg.sv:32-33`, `research-ip/pumice-ddr2-lpddr2/rtl/includes/pumice_pkg.sv:30-31`). Therefore `mc_common_pkg` exports every family-shared symbol the rocks need, but the rocks do not source it directly. `pumice_pkg` additionally exports pumice-only symbols (`addr_map_scheme_e`, `odt_rule_e`) that the common package does not and should not carry.

This is a stable, intentional shim: it leaves rock `import` statements untouched while making the common package the single source of family-shared encodings.

5 mc_axi4_plane — AXI4 host interface and read-return ring

Module: `macro/mc_axi4_plane.sv` **Child FUBs:** `fub/mc_wr_splitter.sv`, `fub/mc_axi_burst_chopper.sv`, `fub/mc_wr_intake.sv`, `fub/mc_rd_intake.sv`, `fub/mc_rd_return_ring.sv`

`mc_axi4_plane` is the host-facing AXI4 slave boundary of the common memory-controller stack. It transforms host AXI4 read and write bursts into DFI-burst-sized descriptors that cross into `mc_storage_plane`, sources read-your-write snarf data from the write-data CAM, and presents the final AXI4 read-data and write-response channels to the host (`macro/mc_axi4_plane.sv:8-12`). It is the N=1 provider of the descriptor seam in lite builds (see common_has chapter 2).



mc_axi4_plane block diagram

5.1 Child-instance map

Instance	Module	Parameter bindings in this plane	Role
u_wr_split	mc_wr_splitter	AXI_ID_WIDTH=IW, AXI_ADDR_WIDTH=AW, AXI_DATA_WIDTH=DW, AXI_USER_WIDTH=UW, AXI_BEATS_PER_BURST=AXI_BEATS_PER_BURST	Split host AW/W into DFI-burst sub-bursts and reframe WLAST (macro/mc_axi4_plane.sv:207-256)
u_rd_split	mc_axi_burst_chopper	AXI_ID_WIDTH=IW, AXI_ADDR_WIDTH=AW, AXI_USER_WIDTH=UW, STRB_BYTES=SW, AXI_BEATS_PER_BURST=AXI_BEATS_PER_BURST	Split host AR into DFI-burst sub-commands (macro/mc_axi4_plane.sv:259-296)
u_wr_intake	mc_wr_intake	full address-map set plus AXI_BEATS_PER_BURST	Write descri

Instance	Module	Parameter bindings in this plane	Role
			ptor push + write-data pop + commit-driven B (`macro/mc_axi4_plane.sv:304-370`)
`u_rd_intake`	`mc_rd_intake`	as `u_wr_intake`, plus `ORDER_FIFO_DEPTH=RD_RET_DEPTH+8`	Read descriptor push + snarf probe + R return (`macro/mc_axi4_plane.sv:373-445`)
`u_rd_ring`	`mc_rd_return_ring`	`DEPTH=RD_RET_DEPTH`, `AXI_DATA_WIDTH=DW`, `AXI_BEATS_PER_BURST=AXI_BEATS_PER_BURST`	In-flight read return buffer (`macro/mc_a`

Instance	Module	Parameter bindings in this plane	Role
			xi4_plane.sv:450-475)

5.2 Burst chop, split, and final coalescing

mc_axi_burst_chopper is an FSM-free AR/AW burst chopper with one active bit, a remaining-beats counter, and a running address. Each sub-command spans at most AXI_BEATS_PER_BURST AXI beats; addresses step by AXI_BURST_BYTES so each sub-command lands on a DRAM-burst boundary (fub/mc_axi_burst_chopper.sv:8-14, fub/mc_axi_burst_chopper.sv:44-64). A ragged final chunk takes min(remaining, AXI_BEATS_PER_BURST) beats (fub/mc_axi_burst_chopper.sv:27-29).

mc_wr_splitter wraps the same chopper for AW with PAD_TO_CHUNK=1 and reframes the W stream so every sub-command sees exactly one WLAST at its AXI_BEATS_PER_BURST boundary (fub/mc_wr_splitter.sv:8-12). The WLAST re-frame is driven by a W-beat down-counter, not by the AW FSM, so it stays exact for any AXI_BEATS_PER_BURST including 1 (fub/mc_wr_splitter.sv:15-22). Short write bursts are completed with zero-strobe filler beats; strb=0 becomes DM=1 in the DFI write serializer (fub/mc_wr_splitter.sv:134-139).

Both splitters emit an aggregation sideband: m_ax_agg is high when the host command splits into more than one sub-command, and m_ax_last is high on the final sub-command (fub/mc_axi_burst_chopper.sv:31-37). The intakes push these bits through their FIFOs so the return path can collapse per-sub B/RLAST into a single host response without a separate aggregator (macro/mc_axi4_plane.sv:200-204). For reads, the intake forwards only the beats the host requested and drops excess DRAM-burst beats; RLAST is generated on the last requested beat, and for a split burst it is asserted only when w_head_last is also set (fub/mc_rd_intake.sv:380-384, fub/mc_rd_intake.sv:465-473).

5.3 Read return ring: ordering contract

`mc_rd_return_ring` separates the in-flight read count from the read-command CAM depth. The CAM entry lives from insert to issue only; after issue, a ticket follows the read through DRAM (`fub/mc_rd_return_ring.sv:8-12`). The ordering rules are:

1. **Allocation order equals AR order.** A ticket is allocated at the ring tail when `alloc_valid_i` is asserted with the CAM insert (`fub/mc_rd_return_ring.sv:53-55`, `fub/mc_rd_return_ring.sv:97-100`).
2. **DFI returns are accepted in issue order**, mapped by an internal ticket FIFO (`fub/mc_rd_return_ring.sv:60-63`, `fub/mc_rd_return_ring.sv:106-120`).
3. **Drain order equals ring/AR order.** The ring drains from its head once the head slot's data is complete; because allocation order is AR order, per-ID AXI reads return in request order structurally (`fub/mc_rd_return_ring.sv:74-80`, `fub/mc_rd_return_ring.sv:135-143`).
4. **A slot is freed only when its last beat is consumed** by the drain (`fub/mc_rd_return_ring.sv:185-186`).

The ring is sized by `RD_RET_DEPTH` (default 32). This decoupling fixed a bandwidth ceiling: with the CAM holding reads for the full DRAM round trip, eight entries over a ~27-cycle round trip bounded board reads to about 180 MB/s; the ring fix took board reads to ~572 MB/s (`research-ip/pumice-ddr2-lpddr2/docs/pumice_mas/ch02_blocks/22_rd_return_ring.md:32-42`, inherited from the pumice measured history).

5.4 Parameters

Parameter	Default	Description
AXI_ID_WIDTH	8	Host AXI ID width
AXI_ADDR_WIDTH	32	Host AXI address width
AXI_DATA_WIDTH	64	Host AXI data width (and DFI word width in this plane)
AXI_USER_WIDTH	1	Host AXI user width
DRAM_BEAT_WIDTH	64	Retained only to size DRAM_BURST_BYTES; pumice instantiates this with the AXI data width, not the DRAM beat width (macro/mc_axi4_plane.sv:29-34)
NUM_RANKS	1	Rank count passed to the address mapper
NUM_BANKS	8	Bank count passed to the address mapper
ROW_WIDTH	14	Row address width passed to the address mapper
COL_WIDTH	10	Column address width passed to the address mapper
BYTE_OFFSET_WIDTH	3	Byte-offset width passed to the address mapper
BG_WIDTH	1	Bank-group width; HAS_BG=0 keeps generations 2/3 bit-identical (macro/mc_axi4_plane.sv:41-44)
HAS_BG	0	Bank-group enable; andesite passes 1/2 (macro/mc_axi4_plane.sv:41-44)
AXI_BEATS_PER_BURST	4	AXI beats per DFI burst; this is

Parameter	Default	Description
		what pumice passes as BURST_WORDS, not the JEDEC burst length (macro/mc_axi4_plane.sv:45-51)
NUM_ENTRIES	8	Storage/scheduler entry count passed from core
N_SRAM_SLOTS	NUM_ENTRIES	SRAM slot count
N_SCHED_LU	4	Scheduler lookup port count
AGE_WIDTH	16	Age field width
RD_RET_DEPTH	32	In-flight read count (ring depth)
IW, AW, DW, UW, SW, BKW, PTRW, RKW, DRAM_BURST_BYTES	derived	Shorthand widths; DRAM_BURST_BYTES = AXI_BEATS_PER_BURST * (DRAM_BEAT_WIDTH / 8)

5.5 Ports

5.5.1 Clock, reset, address map, and status

Port	Direction	Width	Description
`aclk`	input	1	Plane clock
`aresetn`	input	1	Active-low reset
`bank_lsb_i`	input	5	Address-map bank LSB selector
`hash_en_i`	input	1	Bank hash enable
`hash_seed_i`	input	8	Bank hash seed
`busy_o`	output	1	OR of write-intake, read-intake, and return-ring busy flags (`macro/mc_axi4_plane.sv:477`)

5.5.2 Host AXI4 slave

Channel	Port	Direction	Width	Description
AW	`s_axi_awid`	input	`IW`	Write address ID
AW	`s_axi_awaddr`	input	`AW`	Write address
AW	`s_axi_awlen`	input	8	Burst length
AW	`s_axi_awsize`	input	3	Burst size
AW	`s_axi_awburst`	input	2	Burst type
AW	`s_axi_awlock`	input	1	Lock signal
AW	`s_axi_awcache`	input	4	Cache signal
AW	`s_axi_awprot`	input	3	Protection signal

Channel	Port	Direction	Width	Description
AW	s_axi_awqos	input	4	QoS signal
AW	s_axi_awregion	input	4	Region signal
AW	s_axi_awuser	input	UW	User sideband
AW	s_axi_awvalid	input	1	Write address valid
AW	s_axi_awready	output	1	Write address ready
W	s_axi_wdata	input	DW	Write data
W	s_axi_wstrb	input	SW	Write strobe
W	s_axi_wlast	input	1	Write data last
W	s_axi_wuser	input	UW	Write data user
W	s_axi_wvalid	input	1	Write data valid
W	s_axi_wready	output	1	Write data ready
B	s_axi_bid	output	IW	Write response ID
B	s_axi_bresp	output	2	Write response
B	s_axi_buser	output	UW	Write response user
B	s_axi_bvalid	output	1	Write response valid

Channel	Port	Direction	Width	Description
B	s_axi_bready	input	1	Write response ready
AR	s_axi_arid	input	IW	Read address ID
AR	s_axi_araddr	input	AW	Read address
AR	s_axi_arlen	input	8	Burst length
AR	s_axi_arsize	input	3	Burst size
AR	s_axi_arburst	input	2	Burst type
AR	s_axi_arlock	input	1	Lock signal
AR	s_axi_arcache	input	4	Cache signal
AR	s_axi_arprot	input	3	Protection signal
AR	s_axi_arqos	input	4	QoS signal
AR	s_axi_arregion	input	4	Region signal
AR	s_axi_aruser	input	UW	User sideband
AR	s_axi_arvalid	input	1	Read address valid
AR	s_axi_arready	output	1	Read address ready
R	s_axi_rid	output	IW	Read data ID
R	s_axi_rdata	output	DW	Read data

Channel	Port	Direction	Width	Description
R	s_axi_rresp	output	2	Read response
R	s_axi_rlast	output	1	Read data last
R	s_axi_ruser	output	UW	Read data user
R	s_axi_rvalid	output	1	Read data valid
R	s_axi_rready	input	1	Read data ready

5.5.3 Downstream interfaces

Group	Port	Direction	Width	Description
WR push	aw_push_valid_o	output	1	Write command valid
WR push	aw_push_ready_i	input	1	Write command ready
WR push	aw_push_bank_o	output	BKW	Decoded bank
WR push	aw_push_row_o	output	ROW_WIDTH	Decoded row
WR push	aw_push_col_o	output	COL_WIDTH	Decoded column
WR push	aw_push_id_o	output	IW	Write AXI ID
WR push	aw_push_qos_o	output	4	Write QoS
WR push	aw_push_agg_o	output	1	Part of a split host burst
WR push	aw_push_last_o	output	1	Final sub-command of host burst
WR data	wd_valid_o	output	1	Write data valid
WR data	wd_ready_i	input	1	Write data ready

Group	Port	Direction	Width	Description
WR data	wd_data_o	output	DW	Write data
WR data	wd_strb_o	output	SW	Write strobe
WR data	wd_last_o	output	1	Write data last
WR done	wr_done_valid_i	input	1	Write commit completion
WR done	wr_done_id_i	input	IW	Completion ID
Snarf probe	snarf_probe_valid_o	output	1	Probe valid
Snarf probe	snarf_probe_bank_o	output	BKW	Probe bank
Snarf probe	snarf_probe_row_o	output	ROW_WIDTH	Probe row
Snarf probe	snarf_probe_col_o	output	COL_WIDTH	Probe column
Snarf probe	snarf_probe_id_o	output	IW	Read AXI ID
Snarf probe	snarf_probe_len_o	output	8	Read AXI length
Snarf hit	snarf_hit_i	input	1	Probe hit (combinational, one cycle delayed by the CAM register)
Snarf hit	snarf_accept_o	output	1	AR admitted as a snarf hit
Snarf data	snarf_rd_valid_i	input	1	Snarf data valid
Snarf data	snarf_rd_ready_o	output	1	Snarf data ready

Group	Port	Direction	Width	Description
Snarf data	snarf_rd_data_i	input	DW	Snarf data
Snarf data	snarf_rd_last_i	input	1	Snarf data last
RD push	ar_push_valid_o	output	1	Read command valid
RD push	ar_push_ready_i	input	1	Read command ready
RD push	ar_push_bank_o	output	BKW	Decoded bank
RD push	ar_push_row_o	output	ROW_WIDTH	Decoded row
RD push	ar_push_col_o	output	COL_WIDTH	Decoded column
RD push	ar_push_id_o	output	IW	Read AXI ID
RD push	ar_push_qos_o	output	4	Read QoS
Ring alloc	rt_alloc_ready_o	output	1	Ring has space to allocate a ticket
Ring alloc	rt_alloc_ticket_o	output	$clog2(RD_RET_DEPTH)	Allocated ticket (ring tail)
Ring issue	rd_iss_ready_o	output	1	Ring ready for issue notify
Ring issue	rd_iss_valid_i	input	1	Issue notify valid
Ring issue	rd_iss_ticket_i	input	$clog2(RD_RET_DEPTH)	Ticket that issued
Ring gate	rd_cam_ins_ready_i	input	1	Combined gate for ring allocation with ar_push_valid_o (macro/mc_axi4_plane.sv:457)

Group	Port	Direction	Width	Description
DFI return	rd_dfi_ret_valid_i	input	1	DFI return data valid
DFI return	rd_dfi_ret_ready_o	output	1	DFI return data ready
DFI return	rd_dfi_ret_data_i	input	DW	DFI return data
DFI return	rd_dfi_ret_resp_i	input	2	DFI return response
DFI return	rd_dfi_ret_last_i	input	1	DFI return last beat

5.6 What this plane does not know

- **DRAM timing.** No tRAS, tRCD, CAS latency, refresh, or power-state logic resides here.
- **Scheduling.** The plane produces descriptor pushes and accepts scheduler handshakes through the storage plane; it does not pick oldest entries or arbitrate banks.
- **DFI protocol generation.** All DFI command, data, and timing formatting is delegated to the DFI planes.
- **Write-data CAM internals.** The CAM fill, commit, and snarf datapaths are in `mc_storage_plane`; this plane exposes only the raw push, snarf probe, and completion ports.

5.7 Open questions

- Whether `NUM_ENTRIES`, `N_SRAM_SLOTS`, `N_SCHED_LU`, and `AGE_WIDTH` are still consumed by `mc_axi4_plane` itself or are pass-through remnants from an earlier integration; they appear only in the parameter list and do not drive any local logic in the reviewed RTL (`macro/mc_axi4_plane.sv:52-55`).
- Whether `bank_lsb_i` is allowed to place the bank boundary inside the AXI beat; the intakes mask the beat-offset bits before address mapping, but the resulting column alignment behavior for data-width converters wider than the device word is not verified here.

5.8 References

- Smith, Pleszkun, “Implementation of Precise Interrupts in Pipelined Processors,” IEEE Trans. Computers 1988 (orig. ISCA 1985). DOI (conceptual convergence: reorder buffer)

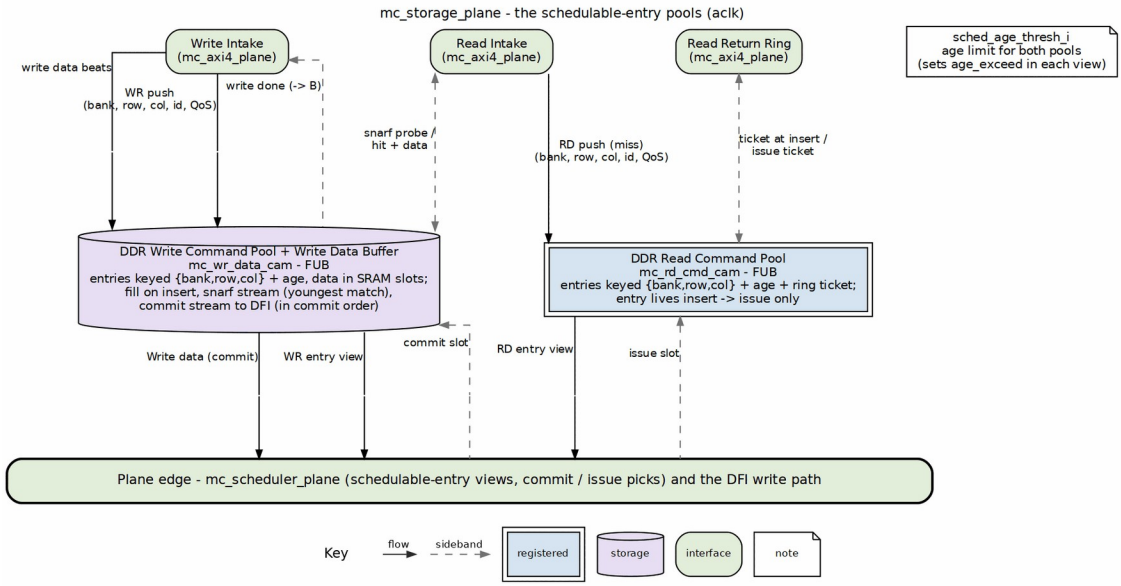
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 mc_storage_plane — Storage Plane and Entry-Pool FUBs

Files: `macro/mc_storage_plane.sv`, `fub/mc_wr_data_cam.sv`, `fub/mc_rd_cmd_cam.sv`, `fub/mc_addr_mapper.sv`

The storage plane holds the family's schedulable-entry pools: the write-data CAM (write buffer + snarf source) and the read-command CAM (read scheduling window). It presents the scheduler-facing bank/row/col image and accepts the read-snoop probe from the upstream AXI4 plane. Within the eight-plane family stack it is the N=8 full provider of the descriptor seam; the stack architecture is documented in common_has.



mc_storage_plane internal block diagram

6.1 mc_storage_plane parameters

Parameter	Default	Description
AXI_ID_WIDTH	8	AXI ID width (IW)
AXI_DATA_WIDTH	64	AXI data width (DW)
NUM_RANKS	1	Rank count
NUM_BANKS	8	Bank count (BKW = $clog2)
ROW_WIDTH	14	Row field width
COL_WIDTH	10	Column field width
AXI_BEATS_PER_BURST	4	Beats per AXI burst
NUM_ENTRIES	8	CAM / scheduling-window depth (PTRW = $clog2)
N_SRAM_SLOTS	NUM_ENTRIES	Write-data SRAM slots
N_SCHED_LU	4	Parallel scheduler lookup ports
AGE_WIDTH	16	Free-running age width
RD_RET_DEPTH	32	Return-ring depth / ticket space; power of 2
IW	AXI_ID_WIDTH	Derived alias
DW	AXI_DATA_WIDTH	Derived alias
SW	AXI_DATA_WIDTH/8	Derived strobe width
BKW	$clog2(NUM_BANKS)	Derived bank-index width
PTRW	$clog2(NUM_ENTRIES)	Derived pointer width
RKW	(NUM_RANKS>1) ? ... : 1	Derived rank-index width

Parameter block: macro/mc_storage_plane.sv:22-45.

6.2 mc_storage_plane ports

Group	Signals	Direction	Width	Description
Clock/reset	aclk, aresetn	input	1, 1	Clock, active-low reset
WR intake cmd	aw_push_valid_i, aw_push_ready_o	in / out	1, 1	AW push handshake
WR intake cmd	aw_push_bank_i, aw_push_row_i, aw_push_col_i, aw_push_id_i, aw_push_qos_i, aw_push_agg_i, aw_push_last_i	input	BKW, ROW_WIDTH, COL_WIDTH, IW, 4, 1, 1	Decoded key, ID, QoS, aggregate, last
WR intake data	wd_valid_i, wd_ready_o, wd_data_i, wd_strb_i, wd_last_i	in / out / in / in / in	1, 1, DW, SW, 1	W-data handshake, data, strobes, last
WR done	wr_done_valid_o, wr_done_id_o	output	1, IW	Commit-done B response
RD snarf probe	snarf_probe_valid_i, snarf_probe_bank_i, snarf_probe_row_i, snarf_probe_col_i, snarf_probe_id_i, snarf_probe_	in / in / in / in / in / in / out / in	1, BKW, ROW_WIDTH, COL_WIDTH, IW, 8, 1, 1	Read probe key/id/len, hit, accept

Group	Signals	Direction	Width	Description
	len_i, snarf_hit_o, snarf_accept_i			
RD snarf stream	snarf_rd_valid_o, snarf_rd_ready_i, snarf_rd_data_o, snarf_rd_last_o	out / in / out / out	1, 1, DW, 1	Forwarded write-data stream
RD intake cmd	ar_push_valid_i, ar_push_ready_o, ar_push_bank_i, ar_push_row_i, ar_push_col_i, ar_push_id_i, ar_push_qos_i	in / out / in / in / in / in / in	1, 1, BKW, ROW_WIDTH, COL_WIDTH, IW, 4	AR push key/id/QoS
Return ring	rt_alloc_ready_i, rt_alloc_ticket_i, rd_iss_ready_i, rd_iss_valid_o, rd_iss_ticket_o, rd_cam_ins_ready_o	in / in / in / out / out / out	1, $clog2(RD_RET_DEPTH), 1, 1, $clog2(RD_RET_DEPTH), 1	Ticket allocation, issue notify, CAM ready
Scheduler ctrl	sched_age_thresh_i	input	8	Age threshold / 16 (0 = off)

Group	Signals	Direction	Width	Description
Scheduler ctrl	wr_commit_valid_i, wr_commit_ready_o, wr_commit_slot_i	in / out / in	1, 1, PTRW	Write commit
Scheduler ctrl	rd_issue_valid_i, rd_issue_ready_o, rd_issue_slot_i	in / out / in	1, 1, PTRW	Read issue
WR sched vectors	wr_sch_valid_o, wr_sch_bank_o, wr_sch_row_o, wr_sch_col_o, wr_sch_older_o, wr_sch_age_exceed_o, wr_sch_qos_o, wr_sch_head_rel_o	output	NUM_ENTRIES, NUM_ENTRIES*BKW, NUM_ENTRIES*ROW_WIDTH, NUM_ENTRIES*COL_WIDTH, NUM_ENTRIES*NUM_ENTRIES, NUM_ENTRIES, NUM_ENTRIES*4, 16	Per-entry WR scheduling image
RD sched vectors	rd_sch_valid_o, rd_sch_bank_o, rd_sch_row_o, rd_sch_col_o, rd_sch_older_o, rd_sch_age_exceed_o, rd_sch_qos_o, rd_sch_head_rel_o	output	(same as WR)	Per-entry RD scheduling image
WR commit data	wr_cm_rd_valid_o, wr_cm_rd_rea	out / in / out / out / out	1, 1, DW, SW, 1	Commit data stream

Group	Signals	Direction	Width	Description
	dy_i, wr_cm_rd_data_o, wr_cm_rd_strb_o, wr_cm_rd_last_o			
Status	busy_o	output	1	Plane busy

Port block: macro/mc_storage_plane.sv:46-159.

6.3 Internal wiring

`mc_storage_plane` instantiates `mc_wr_data_cam` at `macro/mc_storage_plane.sv:165` and `mc_rd_cmd_cam` at `macro/mc_storage_plane.sv:250`. The legacy `oldest_*` and `sched_lu_*` ports on both CAMs are tied off and pruned by synthesis (`macro/mc_storage_plane.sv:205-207` and `macro/mc_storage_plane.sv:269-284`). A read is admitted only when both the CAM and the return ring have room: `assign ar_push_ready_o = rd_cam_ins_ready_o && rt_alloc_ready_i;` (`macro/mc_storage_plane.sv:247`). The return ring hands out a ticket at insert (`macro/mc_storage_plane.sv:104-105`) that the CAM stores and forwards on issue (`macro/mc_storage_plane.sv:297-299`).

6.4 mc_wr_data_cam — write-data CAM / write buffer

Parameter	Default	Description
NUM_ENTRIES	8	CAM depth (PTRW)
N_SCHED_LU	4	Parallel scheduler lookups
NUM_BANKS	8	Bank count (BKW)
ROW_WIDTH	14	Row width
COL_WIDTH	10	Column width
AXI_ID_WIDTH	8	AXI ID width (IW)
AXI_DATA_WIDTH	64	Data width (DW)
AXI_BEATS_PER_BURST	4	Beats per burst
AGE_WIDTH	16	Age-counter width
N_SRAM_SLOTS	NUM_ENTRIES	SRAM data slots
WR_DRAIN_AHEAD	2	Bursts allowed in drain queue
IW / DW / SW / BKW / PTRW / SPTRW / BCW	derived	Width aliases

Parameter block: fub/mc_wr_data_cam.sv:24-45.

Signals	Direction	Width	Description
aclk, aresetn	input	1, 1	Clock, reset
ins_valid_i, ins_ready_o	in / out	1, 1	Insert handshake
ins_bank_i, ins_row_i, ins_col_i, ins_id_i, ins_qos_i, ins_agg_i, ins_last_i	input	BKW, ROW_WIDTH, COL_WIDTH, IW, 4, 1, 1	Command key/id/QoS/aggregate/last
wd_valid_i, wd_ready_o, wd_data_i, wd_strb_i,	in / out / in / in / in	1, 1, DW, SW, 1	Fill data handshake and beat

Signals	Direction	Width	Description
wd_last_i			
snarf_probe_valid_i, snarf_probe_bank_i, snarf_probe_row_i, snarf_probe_col_i, snarf_probe_id_i, snarf_probe_len_i	input	1, BKW, ROW_WIDTH, COL_WIDTH, IW, 8	Read probe
snarf_hit_o, snarf_accept_i	out / in	1, 1	Hit / accepted
snarf_rd_valid_o, snarf_rd_ready_i, snarf_rd_data_o, snarf_rd_last_o	out / in / out / out	1, 1, DW, 1	Snarf data stream
oldest_valid_o, oldest_bank_o, oldest_row_o, oldest_col_o, oldest_id_o, oldest_slot_o	output	1, BKW, ROW_WIDTH, COL_WIDTH, IW, PTRW	Oldest schedulable entry
sched_lu_valid_i, sched_lu_bank_i, sched_lu_row_i	input	N_SCHED_LU, N_SCHED_LU*BKW, N_SCHED_LU*ROW_WIDTH	Scheduler lookup requests
sched_lu_hit_o, sched_lu_slot_o, sched_lu_col_o, sched_lu_id_o, sched_lu_age_o	output	N_SCHED_LU, N_SCHED_LU*PTRW, N_SCHED_LU*COL_WIDTH, N_SCHED_LU*IW, N_SCHED_LU*AGE_WIDTH	Scheduler lookup results
sch_valid_o,	out / out / out / out	NUM_ENTRIES,	Per-entry

Signals	Direction	Width	Description
sch_bank_o, sch_row_o, sch_col_o, sch_older_o, age_thresh_i, sch_age_exceed_o, sch_qos_o, sch_head_rel_o	/ out / in / out / out / out	NUM_ENTRIES*BKW, NUM_ENTRIES*ROW_WIDTH, NUM_ENTRIES*COL_WIDTH, NUM_ENTRIES*NUM_ENTRIES, 8, NUM_ENTRIES, NUM_ENTRIES*4, AGE_WIDTH	scheduling image
commit_valid_i, commit_ready_o, commit_slot_i	in / out / in	1, 1, PTRW	Commit handshake
cm_rd_valid_o, cm_rd_ready_i, cm_rd_data_o, cm_rd_strb_o, cm_rd_last_o	out / in / out / out / out	1, 1, DW, SW, 1	Commit data stream
commit_done_valid_o, commit_done_id_o	output	1, IW	B-response strobe
busy_o	output	1	CAM busy

Port block: fub/mc_wr_data_cam.sv:47-153.

The per-entry state is r_valid, r_bank, r_row, r_col, r_id, r_qos, r_age, r_ptr (SRAM slot), r_pv, r_fdone (fill complete), r_sched (committed), r_agg, r_last, plus the age-order matrix r_older (fub/mc_wr_data_cam.sv:158-189). The burst data lives in one BRAM word per (slot, beat) holding {strb, data}, packed as SRW = DW + SW (fub/mc_wr_data_cam.sv:212-225). A single read port is shared by the snarf and commit streams; a 2-deep prefetch skid decouples fetch from consume (fub/mc_wr_data_cam.sv:228-247). Commit has priority, but snarf anti-starvation caps priority preemption at SN_STARVE_LIMIT = 16 cycles (fub/mc_wr_data_cam.sv:249-258).

Read snooping against the un-coalesced write pool (the snarf path) is limited to the safe case documented in the RTL: a read may snarf a write only when (1) the write is not yet scheduled (!r_sched), (2) the AXI IDs match, and (3) the read burst length equals the write burst length, which reduces to arlen == AXI_BEATS_PER_BURST - 1 because every admitted write is exactly AXI_BEATS_PER_BURST beats (fub/mc_wr_data_cam.sv:314-325). The match also requires r_fdone; among candidates the **youngest** match wins, computed from the age-order matrix rather than a 16-bit relative-age reduction (fub/mc_wr_data_cam.sv:340-364).

The scheduler lookup ports return the **oldest** {bank, row} match (max rel) among entries that are valid, fill-complete, and not yet scheduled (fub/mc_wr_data_cam.sv:390-414), and the per-entry scheduling vectors expose the same qualified state plus the age-order matrix (fub/mc_wr_data_cam.sv:451-466). The oldest-schedulable pick was replaced by the age-order matrix for timing: the RTL records that the previous 16-bit age-reduce path was the worst path in the design after the mode work, measuring 63.6 ns against a 15 ns period, 89 logic levels, 30 carry chains, and 17.8 ns of logic alone — pumice ISSUE-005 (was PUMICE-017) (fub/mc_wr_data_cam.sv:416-426).

Commit is rate-matched: WR_DRAIN_AHEAD = 2 bounds how far the command stream can run ahead of the data drain. The RTL comments note that the old FIFO-room gate allowed a command to run ~20 aclk ahead and produced a measured 29373-cycle DFI hold (fub/mc_wr_data_cam.sv:507-524). Entry eviction happens at fetch-last so the slot frees before the DFI serialization wait (fub/mc_wr_data_cam.sv:754-769).

6.5 mc_rd_cmd_cam — read-command scheduling window

Parameter	Default	Description
NUM_ENTRIES	8	Scheduling-window depth (PTRW)
N_SCHED_LU	4	Parallel scheduler lookups
NUM_BANKS	8	Bank count (BKW)
ROW_WIDTH	14	Row width
COL_WIDTH	10	Column width
AXI_ID_WIDTH	8	AXI ID width (IW)
AGE_WIDTH	16	Age-counter width
RD_RET_DEPTH	32	Return-ring ticket space (TW)
IW / BKW / PTRW / TW	derived	Width aliases

Parameter block: fub/mc_rd_cmd_cam.sv:33-46.

Signals	Direction	Width	Description
aclk, aresetn	input	1, 1	Clock, reset
ins_valid_i, ins_ready_o	in / out	1, 1	Insert handshake
ins_bank_i, ins_row_i, ins_col_i, ins_id_i, ins_qos_i, ins_ticket_i	input	BKW, ROW_WIDTH, COL_WIDTH, IW, 4, TW	Read key/id/QoS + ring ticket
sched_lu_valid_i, sched_lu_bank_i, sched_lu_row_i	input	N_SCHED_LU, N_SCHED_LU*BKW, N_SCHED_LU*ROW_WIDTH	Scheduler lookup requests
sched_lu_hit_o, sched_lu_slot_o, sched_lu_col_o, sched_lu_id_o, sched_lu_age_o	output	N_SCHED_LU, N_SCHED_LU*PTRW, N_SCHED_LU*COL_WIDTH, N_SCHED_LU*IW,	Scheduler lookup results

Signals	Direction	Width	Description
		N_SCHED_LU*AGE_WIDTH	
sch_valid_o, sch_bank_o, sch_row_o, sch_col_o, sch_older_o, age_thresh_i, sch_age_exceed_o, sch_qos_o, sch_head_rel_o	out / out / out / out / out / in / out / out	NUM_ENTRIES, NUM_ENTRIES*BKW, NUM_ENTRIES*ROW_WIDTH, NUM_ENTRIES*COL_WIDTH, NUM_ENTRIES*NUM_ENTRIES, 8, NUM_ENTRIES, NUM_ENTRIES*4, AGE_WIDTH	Per-entry scheduling image
oldest_valid_o, oldest_bank_o, oldest_row_o, oldest_col_o, oldest_id_o, oldest_slot_o	output	1, BKW, ROW_WIDTH, COL_WIDTH, IW, PTRW	Oldest valid entry
issue_valid_i, issue_ready_o, issue_slot_i	in / out / in	1, 1, PTRW	Issue notify
iss_valid_o, iss_ready_i, iss_ticket_o	out / in / out	1, 1, TW	Issue ticket forward
busy_o	output	1	CAM busy

Port block: fub/mc_rd_cmd_cam.sv:48-121.

mc_rd_cmd_cam is the mirror of the write CAM on the read issue side. It stores the read key, ID, QoS, and the ring ticket (fub/mc_rd_cmd_cam.sv:126-133). The entry lives from insert to issue only; the return ring owns the DRAM round trip (fub/mc_rd_cmd_cam.sv:13-18). Until 2026-09-08 this CAM also buffered returned data, which made NUM_ENTRIES the in-flight read limit and capped board read bandwidth at roughly 180 MB/s by Little's law (fub/mc_rd_cmd_cam.sv:20-23). Free-slot allocation is priority-encoded from the high index down (fub/mc_rd_cmd_cam.sv:148-159). Scheduler lookups return the oldest {bank, row} match (fub/mc_rd_cmd_cam.sv:190-212), and the per-entry scheduling vectors plus age-order matrix follow the same idiom as the write CAM (fub/mc_rd_cmd_cam.sv:214-255). On issue the entry is freed and its ticket is forwarded to the return ring (fub/mc_rd_cmd_cam.sv:165-170).

6.6 `mc_addr_mapper` — flat address to DRAM tuple

Parameter	Default	Description
`AXI_ADDR_WIDTH`	32	Flat byte-address width
`NUM_RANKS`	1	1, 2, or 4
`NUM_BANKS`	8	4 or 8
`ROW_WIDTH`	14	Row width
`COL_WIDTH`	10	Column width
`BYTE_OFFSET_WIDTH`	3	`log2(beat byte size)`
`BG_WIDTH`	1	Bank-group width (DDR4)
`HAS_BG`	0	Enable bank-group field

Parameter block: `fub/mc_addr_mapper.sv:57-67`.

Signal	Direction	Width	Description
`axi_addr_i`	input	`AXI_ADDR_WIDTH`	Flat AXI byte address
`bank_lsb_i`	input	5	Bank-field LSB in word address
`hash_en_i`	input	1	Enable bank XOR-hash
`hash_seed_i`	input	8	XOR-hash seed
`rank_o`	output	`$clog2(max(NR,2))`	Decoded rank
`bank_o`	output	`$clog2(NUM_BANKS)`	Decoded bank
`bg_o`	output	`BG_WIDTH`	Bank group (`'0` when `!HAS_BG`)
`row_o`	output	`ROW_WIDTH`	Decoded row
`col_o`	output	`COL_WIDTH`	Decoded column

Port block: fub/mc_addr_mapper.sv:69-82.

The mapper strips byte offsets to a 32-bit word address and slides a BW-bit bank field through the column region at bank_lsb_i; the column is split into col_lo below the bank and col_hi above it, while row and rank sit at invariant positions (fub/mc_addr_mapper.sv:96-120). bank_lsb_i is clamped to [0, COL_WIDTH] (fub/mc_addr_mapper.sv:102-103). bank_lsb == COL_WIDTH gives ROW_MAJOR placement, bank_lsb == log2(DRAM_BL) gives maximal bank interleaving, and intermediate values give partial interleave. The RTL comment notes that the safe lower bound was measured, not derived, and that two older formulas were wrong in opposite directions — pumice BUG-022 (fub/mc_addr_mapper.sv:36-41).

When hash_en_i is set, each bank bit is folded with row bits and the seed: bank_hashed[i] = bank_raw[i] ^ row[i] ^ row[MID] ^ hash_seed[i] where MID is clamped to ROW_WIDTH-1 (fub/mc_addr_mapper.sv:127-133). HAS_BG adds the bank-group field above bank for generation-4 geometries while keeping generation-2/3 arithmetic bit-identical (fub/mc_addr_mapper.sv:63-65).

mc_addr_mapper was extracted on 2026-10-10 (fub/mc_addr_mapper.sv:43). It is not instantiated in this plane: mc_wr_intake and mc_rd_intake each instantiate one (fub/mc_wr_intake.sv:322, fub/mc_rd_intake.sv:236), so it sits in mc_axi4_plane, and all three research rocks reach it through that plane (pumice_core, scoria_core and andesite_core each instantiate mc_axi4_plane). It is documented here because it produces the {bank,row,col} key the pools are searched on.

6.7 Open questions

- The legacy `sched_lu_*` and `oldest_*` ports on both CAMs are tied off inside `mc_storage_plane`. Will they be removed or reconnected for a future scheduler variant?

6.8 References

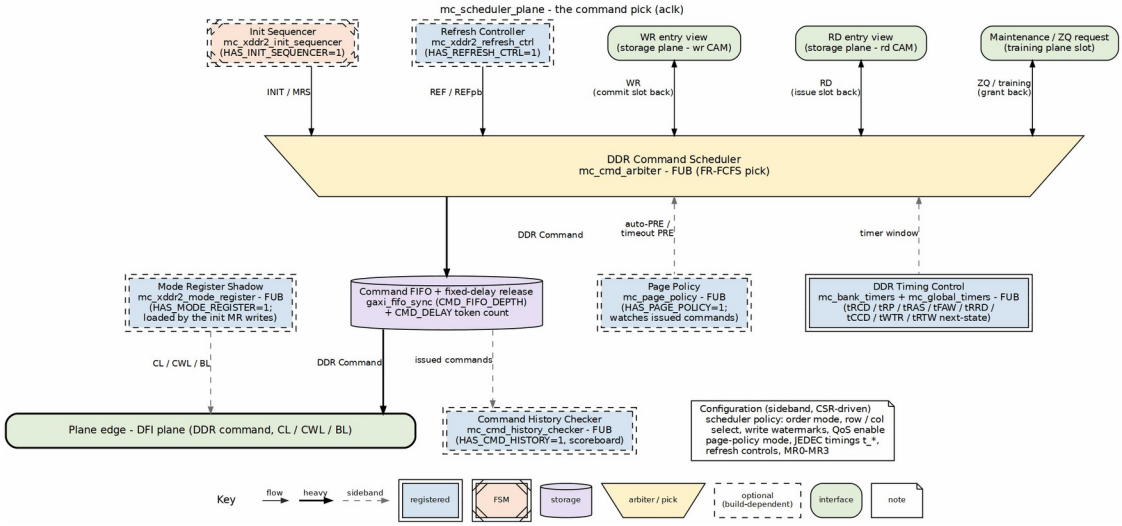
- Tomasulo, “An Efficient Algorithm for Exploiting Multiple Arithmetic Units,” IBM JRD 1967. PDF (conceptual convergence: CAM scoreboard)
- Jacob, Ng, Wang, *Memory Systems: Cache, DRAM, Disk*, Morgan Kaufmann 2007. Publisher page (wr2rd snarf/forwarding; citation-only, book)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Chapter 4 — Scheduler Plane and Pick Machinery

This chapter documents the common `mc_scheduler_plane` and its pick core `mc_cmd_arbiter`. The surrounding plane-stack and descriptor-seam architecture are defined in the Hardware Architecture Specification (HAS); this chapter does not restate them.



mc_scheduler_plane internal block diagram

7.1 1. Top-level parameters

All parameters are declared at `macro/mc_scheduler_plane.sv:27-68`.

Parameter	Default	Notes
`NUM_RANKS`	1	Rank count; drives `RKW` and `NUM_CS`.
`CMD_HISTORY_EN`	0	Arms the command-history checker child (built only when `HAS_CMD_HISTORY` is also 1).
`HIST_T_RCD`, `HIST_T_RP`, `HIST_T_RAS`, `HIST_T_RFC`, `HIST_T_WTR`, `HIST_T_RRD`, `HIST_T_FAW`, `HIST_T_RTW`	varies	History-checker timing parameters.
`NUM_BANKS`	8	Banks per rank; drives `BKW`.
`NUM_BG`	1	Bank groups; `>1` enables DDR4 L/S windows and `cmd_bg_o`.
`ROW_WIDTH`	14	Row-address width.
`COL_WIDTH`	10	Column-address width.
`AXI_ID_WIDTH`	8	AXI ID width; drives `IW`.
`NUM_ENTRIES`	8	CAM entries per direction; drives `PTRW`.
`AGE_WIDTH`	16	Age-counter width.
`CMD_FIFO_DEPTH`	16	Scheduler-to-DFI command FIFO depth.
`CMD_DELAY`	6	Fixed token delay between FIFO push and `cmd_valid_o`.
`HAS_ZQ`	1'b0	Include ZQ calibration branch and tZQCS block.
`MAINT_EXTRA_WIDTH`	1	Width of `cmd_extra_o` / `maint_extra_i`

Parameter	Default	Notes
		(pumice=MRR, andesite=MPC[5:0]).
HAS_INIT_SEQUENCER	1'b1	Build the mc_xddr2_init_sequencer child; 0 takes the init stream from the init_*_i ports.
HAS_MODE_REGISTER	1'b1	Build the mc_xddr2_mode_register child (drives cl_o/cwl_o/bl_o).
HAS_REFRESH_CTRL	1'b1	Build the mc_xddr2_refresh_ctrl child; 0 takes refresh from the refresh_*_i ports.
HAS_PAGE_POLICY	1'b1	Build the mc_page_policy child; 0 takes the AP/timeout-PRE controls from the ap_*_i / timeout_pre_*_i ports and ties the page statistics to 0.
HAS_CMD_HISTORY	1'b0	Build the mc_cmd_history_checker child (simulation scoreboard).
REF_HAS_POSTPONE, REF_HAS_PULLIN, REF_HAS_REFPB, REF_HAS_DRAIN_BURST	1'b1	Feature reduction passed to mc_xddr2_refresh_ctrl.
N_LU	NUM_BANKS	Legacy lookup-width hint; kept for port compatibility with the pumice-era CAM lookup interface.

Parameter	Default	Notes
RKW	derived	(NUM_RANKS>1) ? $clog2(NUM_RANKS) : 1.
NUM_CS	NUM_RANKS	Chip-select count for rank-side tracking.
CSW	derived	(NUM_CS>1) ? $clog2(NUM_CS) : 1.
BKW	derived	$clog2(NUM_BANKS).
PTRW	derived	$clog2(NUM_ENTRIES).
IW	derived	AXI_ID_WIDTH.
BGW_EFF	derived	(NUM_BG>1) ? $clog2(NUM_BG) : 1.

7.2 2. Top-level ports

Ports are declared at `macro/mc_scheduler_plane.sv:69-240`.

Group	Signals	Direction	Width / type	Description
Clock/reset	`aclk`, `aresetn`	in	1	MC clock and active-low reset.
Config	`page_policy_i`	in	`page_policy_e`	OPEN / CLOSE / HAPPY_HYBRID selector.
Scheduler CSR	`sched_order_mode_i`, `sched_row_sel_i`, `sched_col_sel_i`, `sched_access_pref_i`	in	2 each	ORDER mode, row/col select, class preference.
Scheduler CSR	`sched_wr_high_wm_i`, `sched_wr_batch_max_i`, `sched_wr_low_wm_i`	in	8 each	Write-batch watermarks and max.
Scheduler CSR	`sched_prio_sub_i`, `sched_qos_en_i`	in	2, 1	Read-vs-write tie-break and QoS enable.
Page-policy CSR	`page_mode_i`, `page_tr_init_i`	in	3, 8	Consumed by the `mc_page_policy` child (`HAS_PAGE_POLICY=1`).
Stall telemetry	`stall_bp_o`, `stall_refresh_o`,	out	32 each	Per-cause stall counters.

Group	Signals	Direction	Width / type	Description
	stall_turnaround_o, stall_tccd_o, stall_actlimit_o, stall_banktimer_o, stall_noreq_o, stall_zq_o			
Page telemetry	stat_page_hit_o, stat_row_hit_o, stat_page_miss_o, stat_page_empty_o, stat_act_o, stat_pre_o, stat_ref_o, stat_ref_busy_o	out	32 / [NUM_BANKS] [32]	Page/policy telemetry.
Timing CSR	memtype_i	in	memtype_e	DRAM generation selector.
Timing CSR	t_rcd_i, t_rp_i, t_ras_i, t_rc_i, t_wr_i, t_rtp_i, t_faw_i, t_rrd_i, t_wtr_i, t_rtw_i, t_ccd_i	in	8 each	Core JEDEC timing.
Timing CSR	t_ccd_l_i, t_ccd_s_i,	in	8 each	DDR4 long/short

Group	Signals	Direction	Width / type	Description
	t_rrd_l_i, t_rrd_s_i			pair timing.
Refresh	refresh_req_i, refresh_drain_i, refresh_kind_i, refresh_bank_i, refresh_grant_o, t_rfc_i, t_rfc_pb_i	in/out	BKW/16/8	Refresh request/grant/recovery; the request ports are used only when HAS_REFRESH_CTRL=0.
ZQ	zq_req_i, zq_grant_o, t_zqcs_i	in/out	1/1/16	ZQ calibration (present when HAS_ZQ=1).
Maintenance	maint_req_i, maint_grant_o, maint_op_i, maint_bank_i, maint_row_i, maint_col_i, maint_extra_i, maint_busy_i	in/out	varies	Generic maintenance/training slot.
Init passthrough	init_done_i, init_cmd_valid_i, init_cmd_op_i, init_cmd_bank_i, init_cmd_row_i	in	varies	External init command stream; used only when HAS_INIT_SEQUENCER=0.
Init / MR child	t_init_wait_i, t_dll_wait_i, t_mrd_wait_i, t_rp_wait_i,	in/out	varies	CSRs and outputs of the built-in mc_xddr2_init_sequencer /

Group	Signals	Direction	Width / type	Description
	t_rfc_wait_i, mr0_i..mr3_i, init_restart_i, dfi_init_start_o, dfi_init_complete_i, init_done_o, cl_o, cwl_o, bl_o			mc_xddr2_mode_register.
Refresh child	t_refi_i, refi_reload_i, refresh_burst_i, ref_postpone_i, ref_pullin_i, ref_mode_i, ref_trefi_pb_i, ref_trfc_pb_i	in	varies	CSRs of the built-in mc_xddr2_refresh_ctrl.
Page-policy engine	ap_mode_en_i, ap_close_i, timeout_pre_req_i, timeout_pre_bank_i	in	1/[NUM_BANKS]/1/[BKW]	External runtime AP and timeout-PRE controls; used only when HAS_PAGE_POLICY=0.
Write CAM	wr_sch_valid_i, wr_sch_bank_i, wr_sch_row_i, wr_sch_col_i, wr_sch_older	in/out	varies	Write schedulable-entry vectors and commit handshake.

Group	Signals	Direction	Width / type	Description
	_i, wr_sch_age_exceed_i, wr_sch_qos_i, wr_sch_head_rel_i, wr_commit_ready_i, wr_commit_valid_o, wr_commit_slot_o			
Read CAM	rd_sch_valid_i, rd_sch_bank_i, rd_sch_row_i, rd_sch_col_i, rd_sch_older_i, rd_sch_age_exceed_i, rd_sch_qos_i, rd_sch_head_rel_i, rd_issue_ready_i, rd_issue_valid_o, rd_issue_slot_o	in/out	varies	Read schedulable-entry vectors and issue handshake.
Command output	cmd_valid_o, cmd_ready_i, cmd_op_o, cmd_rank_o, cmd_bank_o, cmd_bg_o, cmd_row_o, cmd_col_o,	out/in	varies	DFI command descriptor.

Group	Signals	Direction	Width / type	Description
	cmd_ap_o, cmd_extra_o			
Bank state	bank_row_active_o, bank_open_row_o	out	[NUM_RANKS][NUM_BANKS], [NUM_RANKS][NUM_BANKS][ROW_WIDTH-1:0]	Registered bank-state image.
Status	busy_o	out	1	!init_done || refresh_req || cmd_fifo_valid || any_bank_active (internal init/refresh when the children are built).
Pre-FIFO accept	cmd_accept_o, cmd_accept_op_o	out	1, dram_op_e	Same-cycle accept pulse and opcode.

7.3 3. Internal structure

The plane instantiates the blocks at `macro/mc_scheduler_plane.sv:288-772`. The first four rows are the DDR2/LPDDR2-generation (“v2”) children, each behind its `HAS_*` selector; with the selector at 0 the plane takes that function from its external ports instead. Until 2026-10-11 these children lived in pumice’s `pumice_scheduler_plane` wrapper; they are now common-ip FUBs composed here, and pumice instantiates this plane directly with the defaults.

Instance	Module	Built when	Key parameters	Role
`u_init`	`mc_xddr2_init_sequencer`	`HAS_INIT_SEQUENCER`	`NUM_BANKS`, `ROW_WIDTH`	DDR2/LPDDR2 JEDEC init; drives `dfi_init_start_o` and the init command stream; produces `init_done`.
`u_mode_reg`	`mc_xddr2_mode_register`	`HAS_MODE_REGISTER`	`NUM_RANKS`	MR shadow updated by init writes; decodes `cl_o` / `cwl_o` / `bl_o`.
`u_refresh`	`mc_xddr2_refresh_ctrl`	`HAS_REFRESH_CTRL`	`NUM_BANKS`, `REF_HAS_*`	tREFI accumulator, postpone/pull-in, REFab/REFpb selection; enabled after `init_done`.
`u_page_policy`	`mc_page_policy`	`HAS_PAGE_POLICY`	`NUM_RANKS`, `NUM_BANKS`, `ROW_WIDTH`	Runtime page-policy engine and page telemetry.
`u_bank_timers`	`mc_bank_timers`	always	`NUM_RANKS`, `NUM_BANKS`,	Per-bank safe timers and

Instance	Module	Built when	Key parameters	Role
			ROW_WIDTH, BANK_LA=4	open-row state.
u_global_timers	mc_global_timers	always	NUM_BG, HAS_LS_PAIRS=(NUM_BG>1)	Rank/global windows (tFAW, tRRD, tWTR, tRTW, tCCD, L/S pairs).
u_arbiter	mc_cmd_arbiter	always	NUM_BG, HAS_ZQ, MAINT_EXTRA_WIDTH	Single-issue pick core.
u_cmd_fifo	gaxi_fifo_sync	always	DATA_WIDTH=CMD_W, DEPTH=CMD_FIFO_DEPTH	Scheduler-to-DFI command FIFO.
u_cmd_history	mc_cmd_history_checker	HAS_CMD_HISTORY && CMD_HISTORY_EN != 0	NUM_RANKS, NUM_BANKS, HIST_T_*	Simulation-only JEDEC sequencing scoreboard.

busy_o is !init_done || refresh_req || w_cmd_rd_valid || (|w_bank_row_active[0]) (macro/mc_scheduler_plane.sv:744-745).

7.4 4. Schedulable-entry view and unified prospect pool

The arbiter folds the two CAMs into one `POOL = 2*NUM_ENTRIES` vector: upper half writes, lower half reads (`fub/mc_cmd_arbiter.sv:491-501`). Each entry carries `{valid, bank, row, col}` and is classified as column, activate, or precharge against the registered bank image (`fub/mc_cmd_arbiter.sv:537-589`). Column eligibility further requires row hit, per-bank `bank_rdwr_ready`, the forward tCCD guard, safe turnaround, sink-FIFO room, and clear in-flight guards. This unified-pool refactor (adopted from pumice Task 4.5) is bit-identical to the earlier split loops (`../../mc_scheduler_knobs.md` §4.1).

7.5 5. Generic maintenance slot

Rock-specific training/init/ZQ requesters collapse to a generic maintenance slot: {`maint_req_i, maint_grant_o, maint_op_i, maint_bank_i, maint_row_i, maint_col_i, maint_extra_i, maint_busy_i`} (`macro/mc_scheduler_plane.sv:185-192`). `maint_extra_i` is rock-defined (pumice=MRR, andesite=MPC[5:0]) (`fub/mc_cmd_arbiter.sv:45`; `../../mc_scheduler_knobs.md` §6.3). ZQ keeps its own `zq_req_i`/`zq_grant_o`/`t_zqcs_i` because the tZQCS window must block every other command (`fub/mc_cmd_arbiter.sv:96-101`).

7.6 6. Bank and global timer parameterization

`mc_bank_timers` stamps one `mc_bank_timer` per `(rank,bank)` with advisory lookahead `BANK_LA=4` (`fub/mc_bank_timers.sv:29-35`; `fub/mc_bank_timer.sv:41-44`). Each timer is FSM-free — preset/decrement counters plus a row-open register — so a just-issued command is reflected one cycle later (`fub/mc_bank_timer.sv:15-17`).

`mc_global_timers` exposes DDR4 L/S windows via `NUM_BG`, `BGW`, and `HAS_LS_PAIRS` (`fub/mc_global_timers.sv:33-44`). The plane derives `HAS_LS_PAIRS = (NUM_BG > 1)` and takes the fired command's bank group from the upper bits of `evt_bank` (`macro/mc_scheduler_plane.sv:517-519`). When `HAS_LS_PAIRS=0` the L/S readiness outputs tie to 1 (`fub/mc_global_timers.sv:341-346`).

7.7 7. Output command descriptor

The FIFO write data is packed `{ap, col, row, bg, bank, rank, extra, op}` (`macro/mc_scheduler_plane.sv:697`) and unpacked in the same order at the output (`macro/mc_scheduler_plane.sv:738`). The `extra` field carries the maintenance value for generic maintenance commands; otherwise it is zero. This is the scheduler-side seam; DFI-generation packing is described in DFI Boundary Lineage.

7.8 8. Fire-stage authority

The output stage is the sole timing authority. A registered pick fires only when the live per-bank and global readiness agree:

```
ACT:  bank_act_ready_i && tfaw_ok_i && trrd_ok_i && w_trrd_ls_ok && !
w_zq_busy && !maint_busy_i
RD/WR: bank_rdwr_ready_i && w_tccd_ls_ok && !w_zq_busy && !
maint_busy_i
PRE:  bank_pre_ready_i && !w_zq_busy && !maint_busy_i
```

These terms are at `fub/mc_cmd_arbiter.sv:294-306`. Upstream classify and STAGE-1b masks are advisory.

A pick that fails the live check is normally **dropped**, not held, because CAM commit/issue are qualified by `w_fire_out` and the entry stays schedulable (`fub/mc_cmd_arbiter.sv:252-257`). The single exception is an ACT blocked **only** by the rank-global tFAW/tRRD windows: `w_out_hold` keeps that ACT in the output register (`fub/mc_cmd_arbiter.sv:307-311`). This exception is measured history:

- **pumice BUG-021**: dropping such an ACT regressed close-page throughput from the accepted 30.77% calibration to 18.57%; holding recovered to 30.14% (`fub/mc_cmd_arbiter.sv:259-265`; `../../mc_scheduler_knobs.md` §6.4).
- **scoria BUG-001**: 3/3 seeds failed with a global tRRD violation ("ACT only 1 cyc after another ACT"), leading to the `tfaw_ok_i`/`trrd_ok_i` fire-stage re-check (`fub/mc_cmd_arbiter.sv:267-268`; `../../mc_scheduler_knobs.md` §6.4).
- **ISSUE-002**: moving tFAW/tRRD to the classify mask drained the 3-stage pipeline and dropped close-page throughput from 30.77% to ~18% (`fub/mc_cmd_arbiter.sv:513-518`; `../../mc_scheduler_knobs.md` §6.4).

Other proven guard terms:

- **ONE-CYCLE BLIND SPOT**: turnaround history registers load the cycle after a fire, so a command picked in the same cycle another fires sees stale history. The fix adds an in-flight fire term to the live turnaround equations (`fub/mc_cmd_arbiter.sv:849-858`; `fub/mc_global_timers.sv:100-124`).
- **Parked-victim PRE guard**: a PRE leaves the registered row image stale for up to three cycles; a column picked against it lands on the closed row and wedges the AR-order drain. The column mask includes `!w_pre_col_guard` (`fub/mc_cmd_arbiter.sv:441-453`).

- **BUG-002 / tZQCS fire-stage block**: the tZQCS counter loads on accepted fire, so a classify-time check is one cycle early. Every demand-class fire gate includes `!w_zq_busy` (`fub/mc_cmd_arbiter.sv:278-284`). Refresh is placed above ZQ in the priority cone because inside tZQCS the refresh branch cannot run; `w_ref_safe` carries `!w_rfc_busy`, while refresh is kept out of the ZQ window by cone priority, and the code warns not to “tidy” this into symmetry (`fub/mc_cmd_arbiter.sv:1104-1116`, `1378-1386`).

7.9 9. Page-policy context

The auto-precharge decision is `w_ap = (page_policy_i == PAGE_POLICY_CLOSE)` unless the runtime engine is enabled, in which case it is per-bank via `ap_close_i[b]` (`fub/mc_cmd_arbiter.sv:197-202`). `PAGE_POLICY_HAPPY_HYBRID` was retired and maps to OPEN in the current build (pumice scheduler MAS, lines 200-201). With `HAS_PAGE_POLICY=1` the built-in `mc_page_policy` child supplies the AP-mode enable, per-bank close, and timeout-PRE controls; with it at 0 they come from `ap_mode_en_i`, `ap_close_i`, `timeout_pre_req_i`, and `timeout_pre_bank_i`.

7.10 10. P1 init accept/ack gating

For DDR4 P1 init the product's scheduler wrapper (andesite) must ack its own init sequencer in the same cycle the arbiter accepts the command. Because the arbiter's registered pick re-arms every cycle, a held init request would be picked twice, producing MRS twins one cycle apart. The wrapper gates `init_cmd_valid_i` with a combinational accept pulse from `cmd_accept_o`/`cmd_accept_op_o` (`macro/mc_scheduler_plane.sv:199-200`; `../../../../research-ip/andesite-ddr4-lpddr4/rtl/macro/andesite_scheduler_plane.sv:255-269`; `../../mc_scheduler_knobs.md` §6.4).

7.11 11. Open questions

- The exact encodings of `page_mode_i` and `page_tr_init_i` belong to the `mc_page_policy` child, not to the plane's own logic; this chapter treats them as opaque forwarded CSRs.
- Multi-rank arbitration is fixed at `RK0 = 0` in `mc_cmd_arbiter.sv:194`; whether the same fire-stage authority scales to `NUM_RANKS > 1` is not exercised by the present core.

7.12 References

- Rixner, Dally, Kapasi, Mattson, Owens, “Memory Access Scheduling,” ISCA 2000. PDF
- Mutlu, Moscibroda, “Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors,” MICRO 2007. PDF
- Tomasulo, “An Efficient Algorithm for Exploiting Multiple Arithmetic Units,” IBM JRD 1967. PDF (conceptual convergence: schedulable-entry view)

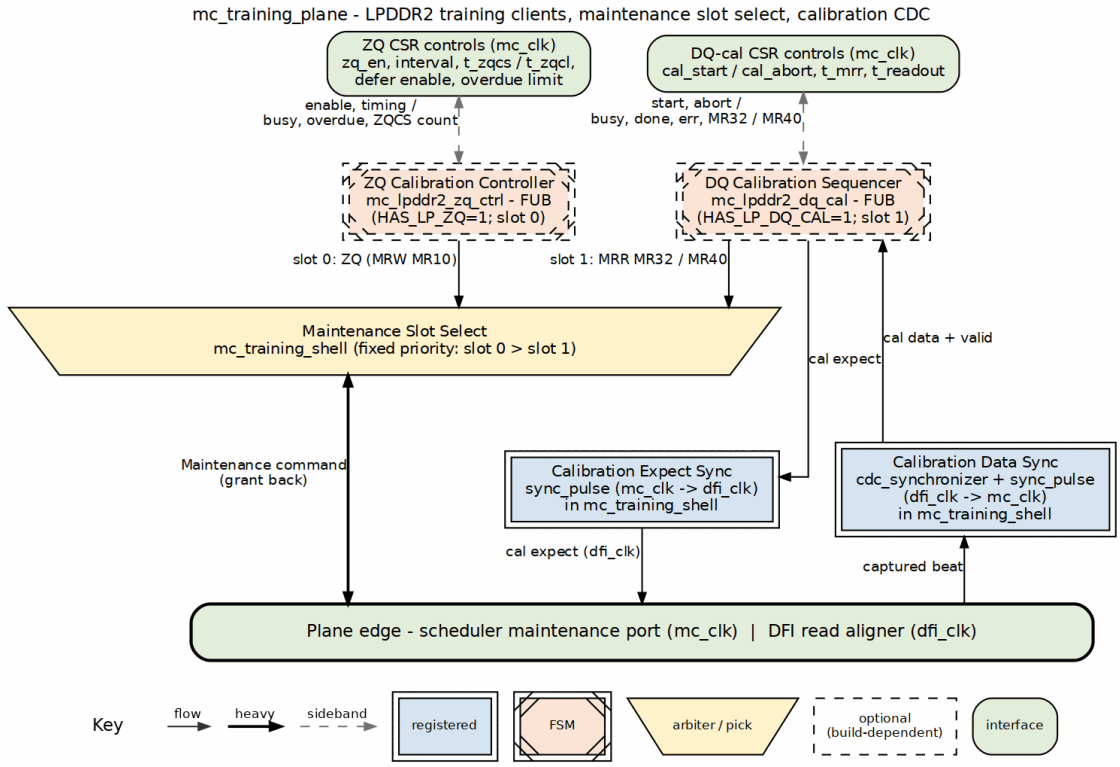
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 mc_training_plane — LPDDR2 training clients on the generic shell

Modules: `mc_training_plane.sv`, `mc_training_shell.sv`
Location: `common-ip/rtl/macro/`
Category: Plane-4 training plane (maintenance-traffic client)
Status: implemented; instantiated directly by `pumice_core` (both clients enabled)

The plane-stack role of this block — where it sits between the scheduler and the PHY calibration sideband — is defined in the family architecture book (common_has). This chapter describes the plane's composition, the generic shell inside it, and how a product configures it.



mc_training_plane block diagram

8.1 Composition

`mc_training_plane` is two layers (`macro/mc_training_plane.sv:7-15`):

- **The LPDDR2 training clients**, each behind a build parameter: `mc_lpddr2_zq_ctrl` (`u_zq`, slot 0, `HAS_LP_ZQ`) and `mc_lpddr2_dq_cal` (`u_lp_cal`, slot 1, `HAS_LP_DQ_CAL`) (`macro/mc_training_plane.sv:112-195`). With a parameter at 0 the client is not elaborated, its slot request is tied low, and its status outputs read 0.
- **`mc_training_shell`** (`u_shell`), the generic, dependency-inverted part: a fixed-priority maintenance-slot bus, a single generic `trn_cmd` output, and the `mc_clk`/`dfi_clk` calibration-data crossing (`macro/mc_training_shell.sv:7-16`). Its `slot_*` ports are internal to the plane; the plane drives them from the clients (`macro/mc_training_plane.sv:200-225`).

The plane, not a product wrapper, owns the slot mapping, the command shapes, and the sideband wiring. A product chooses only parameters. Further training children (CA training, write leveling) would be added as more slots on the shell.

8.1.1 Slot mapping

Slot	Client	Priority	Command shape	`slot_extra` (MRR flag)
0	`mc_lpddr2_zq_ctrl`	highest	MRW to MR10 (`OP_MRS`, row `{4'd0, MR10, OP}`, OP `0x56` ZQCS / `0xAB` ZQCL)	`1'b0`
1	`mc_lpddr2_dq_cal`	lowest	MRR to MR32 then MR40 (`OP_MRS`, row from `mrr_row(MA)`)	`1'b1`

ZQ is periodic maintenance and outranks DQ calibration, which is a firmware-initiated one-shot. For `NUM_SLOTS == 2` the mapping is fixed; for other values the plane assigns ZQ to slot 0 and DQ calibration to slot 1, sets `slot_extra[s] = s[0]`, and leaves any further slots idle (`macro/mc_training_plane.sv:212-223`). The shell's `trn_cmd_extra_o[0]` leaves the plane as `trn_cmd_mrr_o` (`macro/mc_training_plane.sv:265`).

8.1.2 Maintenance-slot arbitration (shell)

The highest-priority requesting slot drives the generic `trn_cmd_*` output to the scheduler; lower-priority slots wait. Priority is fixed, not round-robin: slot 0 > slot 1 > … > slot `NUM_SLOTS-1` (`macro/mc_training_shell.sv:64-74`). Grant is returned only to the selected slot: `slot_grant_o[g] = slot_req_i[g] && trn_cmd_grant_i && (w_sel == g)` (`macro/mc_training_shell.sv:80-85`).

8.1.3 One-active `trn_cmd` mux (shell)

Only one slot's payload reaches the scheduler at a time. The output command is assembled directly from the selected slot's `slot_op_i`, `slot_bank_i`, `slot_row_i`, and `slot_extra_i` (`macro/mc_training_shell.sv:75-78`). The shell never re-encodes the command; the clients produce the opcode, bank, row, and MRR flag.

8.1.4 `mc_clk/dfi_clk` CDC placement (shell)

All calibration-sideband crossing for the plane lives inside the shell, using the repo's standard cells:

- `cal_expect` (mc_clk domain, from `mc_lpddr2_dq_cal`) crosses down to `cal_expect_o` (dfi_clk domain) through `sync_pulse` with three stages (`macro/mc_training_shell.sv:90-97`).
- `cal_data_i` (dfi_clk domain) crosses up to the mc_clk domain through `cdc_synchronizer` with `WIDTH = CAL_DATA_WIDTH` and three flops (`macro/mc_training_shell.sv:99-104`).
- `cal_valid_i` (dfi_clk domain) crosses up to the mc_clk domain through `sync_pulse` with three stages (`macro/mc_training_shell.sv:106-113`).

The synchronized data and valid feed `mc_lpddr2_dq_cal` directly, so both clients run entirely in the mc_clk domain.

8.1.5 Status aggregation

`cal_busy_o = zq_busy_o || dq_cal busy`; `cal_done_o`, `cal_err_o`, `mrr32_data_o` and `mrr40_data_o` come from `mc_lpddr2_dq_cal`; `zq_busy_o`, `zq_overdue_o` and `zqcs_total_o` from `mc_lpddr2_zq_ctrl` (`macro/mc_training_plane.sv:270-274`).

8.2 Boundary doctrine: training searches stay PHY-side

The common training plane owns the JEDEC maintenance-command sequencing and the calibration-control CSR plane. The actual delay-tap, bitslip, and phase-search mechanisms stay on the PHY side of the documented contract (`../../phy_cal_csr_contract.md`). In other words, the core issues ZQ (MRW to MR10) and MRR commands and captures returned beats; firmware plus the PHY walk the taps and compare patterns.

8.3 Parameters

Parameter	Type	Default	Description
NUM_SLOTS	int	2	Number of fixed-priority maintenance slots on the shell. (macro/mc_training_plane.sv:27)
BKW	int	3	Bank field width for the slots and trn_cmd_bank_o. (macro/mc_training_plane.sv:28)
ROW_WIDTH	int	14	Row field width for the slots and trn_cmd_row_o. (macro/mc_training_plane.sv:29)
EXTRA_WIDTH	int	1	Side-band width per slot; bit 0 is the MRR flag. (macro/mc_training_plane.sv:30)
CAL_DATA_WIDTH	int	1	Width of cal_data_i and the captured MRR data; set to the DFI read-data width. (macro/mc_training_plane.sv:31)
HAS_LP_ZQ	bit	1	Build mc_lpddr2_zq_ctrl on slot 0. (macro/mc_training_plane.sv:32)
HAS_LP_DQ_CAL	bit	1	Build mc_lpddr2_dq_cal on slot 1. (macro/mc_training_plane.sv:33)

8.4 Ports

8.4.1 Clock and reset

Signal	Direction	Width	Description
mc_clk	in	1	Controller clock. (macro/mc_training_plane.sv:35)
mc_rst_n	in	1	Active-low reset. (macro/mc_training_plane.sv:36)

8.4.2 Runtime enable / build guard

Signal	Direction	Width	Description
init_done_i	in	1	Training FUBs run only after DRAM init completes. (macro/mc_training_plane.sv:39)
memtype_i	in	memtype_e	DRAM type (DDR2 / LPDDR2). (macro/mc_training_plane.sv:40)

8.4.3 CSR controls

Signal	Direction	Width	Description
zq_en_i	in	1	Enable periodic LPDDR2 ZQ calibration. (macro/mc_training_plane.sv:43)
zq_defer_en_i	in	1	Enable demand-deferred ZQ placement. (macro/mc_training_plane.sv:44)
zq_interval_i	in	32	ZQCS interval in controller clocks.

Signal	Direction	Width	Description
			(macro/mc_training_plane.sv:45)
t_zqcs_i	in	16	ZQCS command post-grant hold time. (macro/mc_training_plane.sv:46)
t_zqcl_i	in	16	ZQCL command post-grant hold time. (macro/mc_training_plane.sv:47)
zq_overdue_max_i	in	13	Overdue-escalation threshold to ZQCL. (macro/mc_training_plane.sv:48)
cal_start_i	in	1	One-shot start for LPDDR2 DQ calibration. (macro/mc_training_plane.sv:50)
cal_abort_i	in	1	Abort in-flight DQ calibration. (macro/mc_training_plane.sv:51)
t_mrr_i	in	16	MRR command interval. (macro/mc_training_plane.sv:52)
t_readout_i	in	16	Readout capture window. (macro/mc_training_plane.sv:53)

8.4.4 Maintenance command channel to the scheduler

Signal	Direction	Width	Description
trn_cmd_req_o	out	1	Aggregate maintenance request. (macro/mc_training_pla

Signal	Direction	Width	Description
			ne.sv:56)
trn_cmd_grant_i	in	1	Arbiter grant. (macro/mc_training_plane.sv:57)
trn_cmd_op_o	out	dram_op_e	Opcode of the selected slot. (macro/mc_training_plane.sv:58)
trn_cmd_bank_o	out	BKW	Bank of the selected slot. (macro/mc_training_plane.sv:59)
trn_cmd_row_o	out	ROW_WIDTH	Row of the selected slot. (macro/mc_training_plane.sv:60)
trn_cmd_mrr_o	out	1	MRR side-band flag to the scheduler. (macro/mc_training_plane.sv:61)

8.4.5 DFI-plane calibration sideband

Signal	Direction	Width	Description
dfi_clk	in	1	DFI/PHY clock. (macro/mc_training_plane.sv:64)
dfi_rstn	in	1	Active-low DFI reset. (macro/mc_training_plane.sv:65)
cal_expect_o	out	1	dfi_clk-domain arm pulse to the PHY/read aligner. (macro/mc_training_plane.sv:66)
cal_data_i	in	CAL_DATA_WIDTH	dfi_clk-domain captured beat. (macro/mc_training_pla

Signal	Direction	Width	Description
			ne.sv:67)
cal_valid_i	in	1	dfi_clk-domain “data landed” pulse. (macro/mc_training_plane.sv:68)

8.4.6 Telemetry / status

Signal	Direction	Width	Description
cal_busy_o	out	1	Either training client is active. (macro/mc_training_plane.sv:71)
cal_done_o	out	1	DQ calibration completed (sticky). (macro/mc_training_plane.sv:72)
cal_err_o	out	1	DQ calibration error (sticky). (macro/mc_training_plane.sv:73)
mrr32_data_o	out	CAL_DATA_WIDTH	Captured MR32 read data. (macro/mc_training_plane.sv:74)
mrr40_data_o	out	CAL_DATA_WIDTH	Captured MR40 read data. (macro/mc_training_plane.sv:75)
zq_busy_o	out	1	ZQ calibration is active. (macro/mc_training_plane.sv:76)
zq_overdue_o	out	1	ZQ is overdue. (macro/mc_training_plane.sv:77)
zqcs_total_o	out	16	Count of completed ZQCS

Signal	Direction	Width	Description
			operations. (`macro/mc_training_plane.sv:78`)

8.5 Product integration

A product instantiates `mc_training_plane` directly and:

1. Chooses its clients with `HAS_LP_ZQ` / `HAS_LP_DQ_CAL`.
2. Sets `BKW`, `ROW_WIDTH`, `EXTRA_WIDTH` to match its scheduler's maintenance slot, and `CAL_DATA_WIDTH` to its DFI read-data width.
3. Connects `trn_cmd_*` to the scheduler's generic maintenance slot (`maint_req_i` / `maint_grant_o` / `maint_op_i` / `maint_bank_i` / `maint_row_i`, with `trn_cmd_mrr_o` on `maint_extra_i[0]` and `cal_busy_o` on `maint_busy_i`).
4. Connects `cal_expect_o` / `cal_data_i` / `cal_valid_i` to the DFI plane's read-aligner calibration sideband, and the CSR controls and status to its register block.

Run gating (`init_done`, `memtype == LPDDR2`) and JEDEC command packing live in the client FUBs, not in the shell, so a DDR2 run leaves the plane inert without a rebuild.

8.5.1 Pumice

`pumice_core` instantiates the plane as `u_train_plane` with `NUM_SLOTS = 2`, `EXTRA_WIDTH = 1`, `CAL_DATA_WIDTH = DFI_DATA_WIDTH`, and both clients enabled (`research-ip/pumice-ddr2-lpddr2/rtl/top/pumice_core.sv:756-797`). Until 2026-10-11 pumice reached the shell through its own `pumice_training_plane` wrapper around pumice-local `pumice_zq_ctrl` / `pumice_lp_cal`; those FUBs became `mc_lpddr2_zq_ctrl` / `mc_lpddr2_dq_cal`, the composition moved into this plane, and the wrapper was removed. The clients' behaviour and measured history are still documented in the pumice MAS (ZQ control, DQ calibration) and the pumice HAS Training chapter.

8.6 ZQ placement as a maintenance client

ZQ calibration is a maintenance-class request: it raises slot 0, waits for its grant, and never preempts an in-flight demand command. The plane preserves the BUG-003 discipline: the scheduler holds demand-class traffic during the post-grant hold (`cal_busy_o` onto `maint_busy_i`, folded into the fire stage) so that `w_fire_out == cmd_valid_o && cmd_ready_i` holds by construction, and the training plane never touches the pick cone (pumice MAS `23_zq_ctrl`).

`mc_lpddr2_zq_ctrl.demand_i` is tied to `1'b0` inside the plane (`macro/mc_training_plane.sv:125`). This means the Mode-C deferral entry condition (`zq_defer_en_i && demand_i`) can never fire, the overdue/ZQCL escalation path is dormant, and `zq_overdue_o` never sets. The FSM keeps the policy; it needs only one wire to make it live.

8.7 Plane-only versus pumice-proven

Capability	Provenance
2-slot fixed-priority arbitration (ZQ > DQ cal)	pumice-proven
Generic `trn_cmd_*` output mapped to the scheduler maintenance slot	pumice-proven
`sync_pulse` + `cdc_synchronizer` calibration sideband crossing	pumice-proven at `CAL_DATA_WIDTH = 128`
`HAS_LP_ZQ = 0` / `HAS_LP_DQ_CAL = 0` builds	plane-only; no product builds them
`NUM_SLOTS > 2`	plane-only; extra slots stay idle
`EXTRA_WIDTH > 1` or multi-bit sideband semantics	plane-only
Re-arbitration/round-robin policy	not implemented
Multi-rank ZQ overlap policy	not implemented; pumice is single-rank
`cal_expect` from more than one client	not implemented; only `mc_lpddr2_dq_cal` drives it

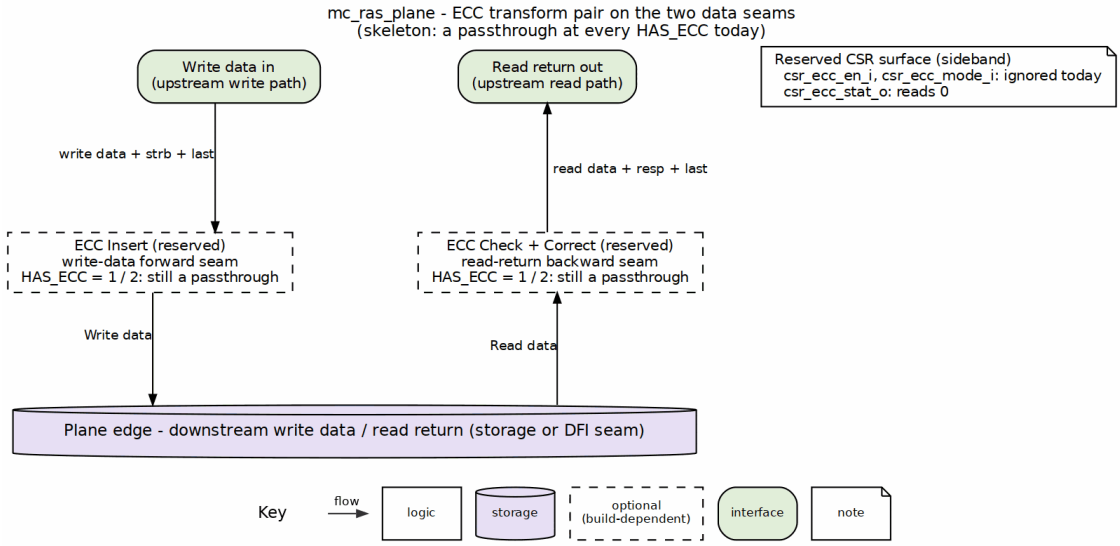
8.8 Open questions

- **Demand wiring for Mode-C ZQ placement.** Should the plane expose a `demand_i` port so the scheduler's demand indicator reaches `mc_lpddr2_zq_ctrl` instead of the internal tie-off?
- **Slot-width defaults.** The clients produce 3-bit bank and 18-bit row fields whose top four row bits are zero, so `ROW_WIDTH` must be at least 14 to carry `{MA[5:0], OP[7:0]}`; the defaults (`BKW = 3`, `ROW_WIDTH = 14`) are exactly that floor. Is the default `EXTRA_WIDTH = 1` still useful once MRR flags graduate to a dedicated scheduler field?
- **Multi-client `cal_expect`.** If the plane ever composes more than one calibration child that needs `cal_expect`, the shell's single `cal_expect_i` must be muxed or widened. No design exists yet.

9 Chapter 6 — mc_ras_plane

mc_ras_plane is the Reliability/Availability/Serviceability transform plane on the two memory-data seams: write-data forward and read-return backward (macro/mc_ras_plane.sv:8-9). The common-ip extraction intentionally leaves it a skeleton; the only binding decisions are the module boundary, the transform-pair placement, the HAS_ECC parameter intent, and a reserved CSR surface. The architecture context is in the family HAS (common_has); this chapter does not restate it.

The plane is not yet instantiated into any research controller (mc_ras_plane_knobs.md).



mc_ras_plane

9.1 Fixed decisions

- **Module boundary.** Two AXI-like streaming seams, one in each direction, plus a four-signal CSR surface and a `busy_o` indicator (`macro/mc_ras_plane.sv:27-69`).
- **Transform-pair placement.** ECC insertion belongs on the write-data forward seam; ECC check belongs on the read-return backward seam. The pair wraps either the storage plane or a DFI plane, depending on where the code word is carried. This placement is the only RAS structure with two customers today (common_has doctrine 9).
- **`HAS_ECC` intent.** `0` = `ECC_OFF` (default and only implemented value), `1` = `ECC_DETECT` reserved, `2` = `ECC_CORRECT` reserved (`macro/mc_ras_plane.sv:22-23`).
- **Reserved CSR surface.** `csr_ecc_en_i`, `csr_ecc_mode_i`, and `csr_ecc_stat_o` exist at the boundary; when `HAS_ECC = ECC_OFF` the status port reads zero (`macro/mc_ras_plane.sv:62-66, 86`). The full field layout is recorded in `mc_ras_plane_knobs.md`.
- **Scrubber relationship.** When added, the scrubber will be a maintenance-class client of the scheduler/prospect pool, peer to refresh and training, injecting read descriptors that reuse existing read-snoop hazard rules (`mc_ras_plane_knobs.md`).

9.2 Parameters

Parameter	Default	Description	RTL
DATA_WIDTH	64	Data beat width on both seams	macro/mc_ras_plane.sv:20
STRB_WIDTH	DATA_WIDTH/8	Byte-strobe width on the write seam	macro/mc_ras_plane.sv:21
HAS_ECC	0	0=OFF, 1=DETECT (reserved), 2=CORRECT (reserved)	macro/mc_ras_plane.sv:23
ECC_MODE	0	Reserved knob for in-band vs. side-band code placement	macro/mc_ras_plane.sv:25
ECC_CODE_WIDTH	8	Reserved knob for ECC code-word width	macro/mc_ras_plane.sv:26

9.3 Ports

9.3.1 Write-data forward seam (upstream → downstream)

Port	Direction	Width	Description	RTL
wdata_valid_i	input	1	Upstream write-data valid	macro/mc_ras_plane.sv:34
wdata_ready_o	output	1	Upstream write-data ready	macro/mc_ras_plane.sv:35
wdata_data_i	input	DATA_WIDTH	Upstream write data	macro/mc_ras_plane.sv:36
wdata_strb_i	input	STRB_WIDTH	Upstream write strobe	macro/mc_ras_plane.sv:37
wdata_last_i	input	1	Upstream write last beat	macro/mc_ras_plane.sv:38
wdata_valid_o	output	1	Downstream write-data valid	macro/mc_ras_plane.sv:40
wdata_ready_i	input	1	Downstream write-data ready	macro/mc_ras_plane.sv:41
wdata_data_o	output	DATA_WIDTH	Downstream write data	macro/mc_ras_plane.sv:42
wdata_strb_o	output	STRB_WIDTH	Downstream write strobe	macro/mc_ras_plane.sv:43
wdata_last_o	output	1	Downstream write last beat	macro/mc_ras_plane.sv:44

9.3.2 Read-return backward seam (downstream → upstream)

Port	Direction	Width	Description	RTL
rdata_valid_i	input	1	Downstream	macro/mc_ras_plane

Port	Direction	Width	Description	RTL
			read-return valid	.sv:49
rdata_ready_o	output	1	Downstream read-return ready	macro/ mc_ras_plane .sv:50
rdata_data_i	input	DATA_WIDTH	Downstream read data	macro/ mc_ras_plane .sv:51
rdata_resp_i	input	2	Downstream read response	macro/ mc_ras_plane .sv:52
rdata_last_i	input	1	Downstream read last beat	macro/ mc_ras_plane .sv:53
rdata_valid_o	output	1	Upstream read-return valid	macro/ mc_ras_plane .sv:55
rdata_ready_i	input	1	Upstream read-return ready	macro/ mc_ras_plane .sv:56
rdata_data_o	output	DATA_WIDTH	Upstream read data	macro/ mc_ras_plane .sv:57
rdata_resp_o	output	2	Upstream read response	macro/ mc_ras_plane .sv:58
rdata_last_o	output	1	Upstream read last beat	macro/ mc_ras_plane .sv:59

9.3.3 Reserved CSR surface

Port	Direction	Width	Description	RTL
csr_ecc_en_i	input	1	Master ECC enable; no effect when HAS_ECC = OFF	macro/ mc_ras_plane .sv:64
csr_ecc_mode_i	input	2	ECC operating mode;	macro/ mc_ras_plane

Port	Direction	Width	Description	RTL
			reserved	.sv:65
csr_ecc_stat_o	output	32	ECC status; reads zero when HAS_ECC = OFF	macro/ mc_ras_plane .sv:66
busy_o	output	1	Plane busy indicator; tied low	macro/ mc_ras_plane .sv:68

9.4 Bypass property

A build that does not need RAS features omits this plane entirely; absence is a true bypass, not a stub. When the plane is present, both `gen_ecc_off` and `gen_ecc_reserved` reduce to bit-identical passthrough assigns (`macro/mc_ras_plane.sv:71-105`). The RTL comment notes that the `HAS_ECC = 0` branch contains “no combinational logic beyond the assigns themselves” (`macro/mc_ras_plane.sv:72-73`).

9.5 Explicitly not decided

Per doctrine 9, any RAS feature beyond the transform pair (ECC insert/check) plus scrub-as-maintenance-client is out of scope until a real enabling customer appears (`common_has` doctrine 9). In particular:

- The ECC encoder, decoder, and syndrome logic are not implemented (`macro/mc_ras_plane.sv:88-90`).
- `ECC_MODE` and `ECC_CODE_WIDTH` are documented reserved knobs but have no RTL effect (`macro/mc_ras_plane.sv:24-26`).
- The scrubber has no implementation; its scheduler interface is deferred to the scheduler-merge task (`mc_ras_plane_knobs.md`).

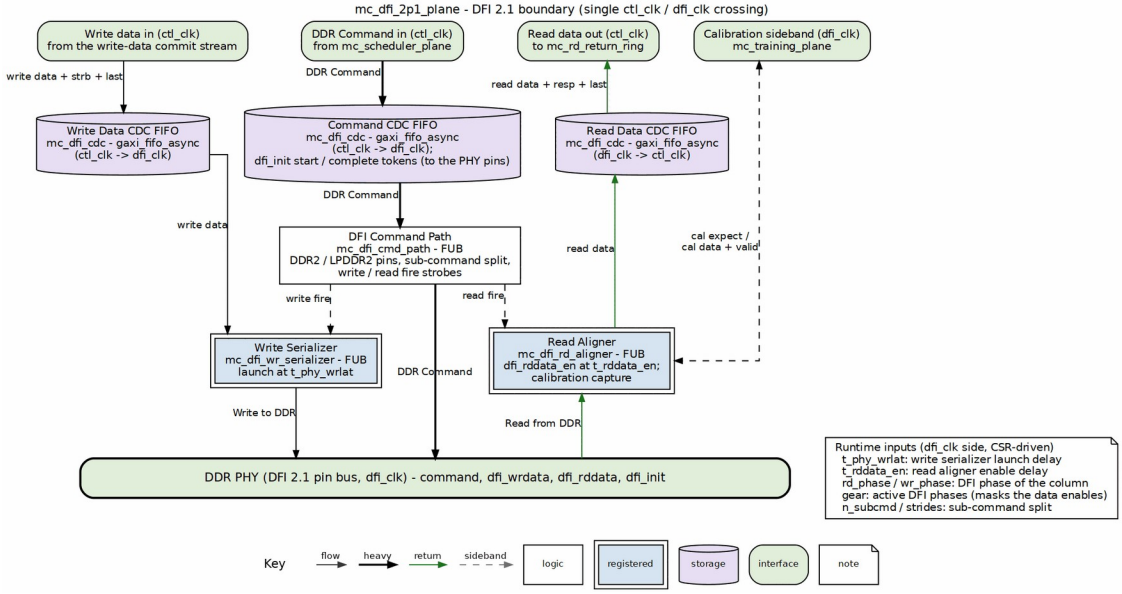
9.6 Open questions

- Which side of the storage/DFI boundary hosts the ECC code word? The transform-pair placement is fixed, but the partner has not selected the code-word carrier.
- What error-response encoding appears on `rdata_resp_o` when `HAS_ECC = DETECT/CORRECT`? The `rdata_resp` width is two bits (`macro/mc_ras_plane.sv:52`), but no encoding is assigned.

10 mc_dfi_2p1_plane — DFI 2.1 Command / Write / Read Pipeline

Module: `mc_dfi_2p1_plane.sv` · **Location:** `common-ip/rtl/macro/`

Architectural context is in the family HAS (common_has); this chapter documents the block microarchitecture only.



mc_dfi_2p1_plane internal block diagram

10.1 Purpose

`mc_dfi_2p1_plane` is the common DFI 2.1 PHY-facing plane. It presents controller-clock command, write-data, and read-data streams on one side and the DFI 2.1 pin bus on the other, and owns the single controller-to-PHY clock-domain crossing plus the DFI-clock command path, write serializer, and read aligner (`macro/mc_dfi_2p1_plane.sv:8-12`, `macro/mc_dfi_2p1_plane.sv:32`). Its datapath unit is the **DFI word**; one FIFO word equals one DFI cycle, so the path is bubble-free at rate (`macro/mc_dfi_2p1_plane.sv:24-25`).

10.2 Sub-FUBs and clock domains

Instance	Domain	Role	File
u_cdc	ctl_clk ↔ dfi_clk	The single family CDC: async gaxi_fifo_async FIFOs for cmd, wrdata, rddata; token crossings for init_start/init_complete	fub/mc_dfi_cdc.sv:30
u_cmd	dfi_clk	Command FIFO → multi-phase DFI command bus; emits wr_fire/rd_fire strobes	fub/mc_dfi_cmd_path.sv:47
u_wr	dfi_clk	Write-data FIFO → dfi_wrdata/dfi_wrdata_en/dfi_wrdata_mask at t_phy_wrlat	fub/mc_dfi_wr_serializer.sv:29
u_rd	dfi_clk	dfi_rddata capture → read-return FIFO at t_rddata_en	fub/mc_dfi_rd_aligner.sv:38

10.3 The single controller↔PHY clock-domain crossing

`mc_dfi_cdc` is the only place the controller and PHY clock domains meet (`fub/mc_dfi_cdc.sv:8-12`). It uses `gaxi_fifo_async` for every crossing — cmd, wrdata, rddata, plus one-bit token FIFOs for `init_start` and `init_complete` — and prohibits hand-rolled synchronizers or open-loop bit crossings (`fub/mc_dfi_cdc.sv:11-12`).

The level signals `init_start_i` and `dfi_init_complete_i` cross as **event tokens**: a rising edge on the source pushes a 1-bit token; the sink pops it into a sticky latch (`fub/mc_dfi_cdc.sv:20-22`, `fub/mc_dfi_cdc.sv:188-248`). The write path also pushes a `pwr_staged` token per complete write burst so the command path cannot issue a WR ahead of its data (`fub/mc_dfi_cdc.sv:86-93`).

10.4 DFI_RATE gearing and bus geometry

DFI_RATE is the compile-time maximum number of DRAM beats per DFI word (phase count). The plane derives the DFI bus widths from it (macro/mc_dfi_2p1_plane.sv:70-82):

Derived width	Formula
DFI_DATA_WIDTH	DRAM_BEAT_WIDTH * DFI_RATE
DFI_STRB_WIDTH	DFI_DATA_WIDTH / 8
DFI_EN_WIDTH	DFI_RATE
DFI_VALID_WIDTH	DFI_RATE
DFI_ADDR_BUS_W	ROW_WIDTH * DFI_RATE
DFI_BANK_BUS_W	$clog2(NUM_BANKS) * DFI_RATE
DFI_CTRL_BUS_W	1 * DFI_RATE
DFI_CS_BUS_W	NUM_RANKS * DFI_RATE

At runtime gear_i selects the active rate within the compile-time maximum: active_rate = 1 << gear_i (macro/mc_dfi_2p1_plane.sv:143-148). The plane builds a per-phase active mask w_phase_active and gates dfi_wrdata_en_o/dfi_rddata_en_o to the active phases (macro/mc_dfi_2p1_plane.sv:202-211). At gear_i = $clog2(DFI_RATE) the mask is all-ones and the outputs are bit-identical to the unmasked FUB outputs.

10.5 Command path

`mc_dfi_cmd_path` pops the abstract command stream from the CDC FIFO, unpacks `{mrr, ap, col, row, bank, rank, op}`, and drives the multi-phase DFI command bus (`fub/mc_dfi_cmd_path.sv:8-10`). It places READ on `rd_phase_i` and WRITE on `wr_phase_i`; everything else lands on phase 0 (`fub/mc_dfi_cmd_path.sv:231-238`).

The path performs **no JEDEC timing enforcement**: all delays come from the scheduler. The only remaining holds are structural — a read may not issue unless the aligner has a free slot, and a write may not issue before its data is staged (`fub/mc_dfi_cmd_path.sv:175-214`). This policy followed pumice `BUG-017` (was `PUMICE-042`): holding columns for `tRTW`/`tWTR` here compressed REF→ACT spacing in the in-order FIFO and caused 180 consecutive reads to return an undriven DQ bus (`fub/mc_dfi_cmd_path.sv:181-190`; see pumice MAS `15_gear_dfi.md`).

For sub-DFI-word packing, `N_SUBCMD` sub-column-commands of one scheduled column command are issued in a single DFI cycle at phases `{base, base+sub_phase_stride_i, ...}` and columns `{col, col+sub_col_stride_i, ...}` (`fub/mc_dfi_cmd_path.sv:216-227`). The per-sub formatter buses merge by OR for address/bank/odt and AND for the active-low control strobes (`fub/mc_dfi_cmd_path.sv:308-332`). One `wr_fire`/`rd_fire` group strobe is emitted per packed group, so the data paths drive/capture the single DFI word once (`fub/mc_dfi_cmd_path.sv:396-411`).

10.6 Write serializer

`mc_dfi_wr_serializer` is stateless. On `wr_fire_i` it matures after `t_phy_wrlat_i` DFI cycles, then streams one DFI word per cycle from the FIFO onto `dfi_wrdata_o` until `wd_last_i` (`fub/mc_dfi_wr_serializer.sv:8-11`, `fub/mc_dfi_wr_serializer.sv:64-75`). It tracks matured-but-not-finished bursts in `r_owed` and drives whenever a burst is owed and the FIFO has a word (`fub/mc_dfi_wr_serializer.sv:91-99`). `dfi_wrdata_mask_o = ~wd_strb_i` while driving (`fub/mc_dfi_wr_serializer.sv:22`, `fub/mc_dfi_wr_serializer.sv:113`).

10.7 Read aligner

mc_dfi_rd_aligner admits a read on op_valid_i && op_ready_o. op_ready_o deasserts when MAX_OUTSTANDING reads are already in flight (fub/mc_dfi_rd_aligner.sv:18-25, fub/mc_dfi_rd_aligner.sv:91-98). On admission it drives dfi_rddata_en_o for EN_CYC cycles starting t_rddata_en_i cycles later (fub/mc_dfi_rd_aligner.sv:100-120). It captures dfi_rddata_i whenever dfi_rddata_valid_i is asserted and a read is outstanding, pushing {last, resp=OKAY, data} into the read FIFO (fub/mc_dfi_rd_aligner.sv:127-166).

The capture uses a combinational enable-window credit to reject the PHY preamble valid before the read window; a registered credit dropped the first word in the a7gated model (fub/mc_dfi_rd_aligner.sv:132-146). The 2.1 plane enables the calibration sideband: cal_expect_i arms a one-shot capture of the first valid beat into cal_data_o/cal_valid_o (fub/mc_dfi_rd_aligner.sv:169-201).

The read-return FIFO must cover the whole read-admission domain; RD_FIFO_DEPTH defaults to 32 because depth 16 dropped beats under a paced-but-adjacent burst schedule (macro/mc_dfi_2p1_plane.sv:55-63).

10.8 PHY timing knobs

Knob	Width	Programmed by	Measured pumice value
t_phy_wrlat_i	[7:0]	PHY_TIMING CSR, bits 7:0	1 on Nexys A7 a7ddrphy (research-ip/pumice-ddr2-lpddr2/docs/pumice_mas/ch04_apb_config/02_csr_map.md:272)
t_rddata_en_i	[7:0]	PHY_TIMING CSR, bits 15:8	6 on Nexys A7 a7ddrphy (research-ip/pumice-ddr2-lpddr2/docs/pumice_mas/ch04_apb_config/02_csr_map.md:273)
rd_phase_i / wr_phase_i	PHW	DFI_PHASE CSR	0 / 0 on Nexys A7 (research-ip/pumice-ddr2-lpddr2/docs/pumice_mas/ch05_programming/04_error_handling.md:67)

These PHY-integration knobs are not datapath constants; pumice is the measured customer (Nexys A7, 66.67 / 75 MHz) (common-ip/docs/common_has/ch02_architecture/02_descriptor_seam.md:97). The board-validated tuple (2026-07-21) is rd_phase=0, t_rddata_en=6, t_phy_wrlat=1, plus harness DFI_TUNING.rddata_delay=7 (research-ip/pumice-ddr2-lpddr2/docs/pumice_mas/ch04_apb_config/03_runtime_overrides.md:97).

10.9 Parameters

Parameter	Default	Description
NUM_RANKS	1	CS/rank fan-out
NUM_BANKS	8	Number of banks
ROW_WIDTH	14	Row / address width
COL_WIDTH	10	Column width
DFI_RATE	2	Compile-time max DRAM beats per DFI word
DRAM_BEAT_WIDTH	64	One DRAM beat data width
DFI_BEATS_PER_BURST	8	DRAM beats per burst
N_SUBCMD	1	Max sub-column-commands packed into one DFI word
SUB_COL_STRIDE	1	Compile-time max device-word column stride
SUB_PHASE_STRIDE	derived	Compile-time max DFI-phase stride between packed sub-bursts
SUBW_MAX	derived	Width holding N_SUBCMD
BURST_WORDS	derived	DFI words per burst-group
CMD_FIFO_DEPTH	8	CDC command FIFO depth
WD_FIFO_DEPTH	16	CDC write-data FIFO depth
RD_FIFO_DEPTH	32	CDC read-return FIFO depth
N_FLOP_CROSS	2	Async-FIFO pointer synchronizer stages
USE_JOHNSON	0	Pointer encoding: 0=Gray, 1=Johnson
DFI_DATA_WIDTH	derived	DRAM_BEAT_WIDTH * DFI_RATE
DFI_STRB_WIDTH	derived	DFI_DATA_WIDTH / 8
DFI_EN_WIDTH	derived	DFI_RATE
DFI_VALID_WIDTH	derived	DFI_RATE
DFI_ADDR_WIDTH	derived	ROW_WIDTH
DFI_BANK_WIDTH	derived	$clog2(NUM_BANKS)

Parameter	Default	Description
DFI_CTRL_WIDTH	1	Control strobe width
DFI_CS_WIDTH	derived	NUM_RANKS
DFI_ADDR_BUS_W	derived	DFI_ADDR_WIDTH * DFI_RATE
DFI_BANK_BUS_W	derived	DFI_BANK_WIDTH * DFI_RATE
DFI_CTRL_BUS_W	derived	DFI_CTRL_WIDTH * DFI_RATE
DFI_CS_BUS_W	derived	DFI_CS_WIDTH * DFI_RATE
RKW	derived	Rank index width
BKW	derived	Bank index width
PHW	derived	Phase index width
CMD_HISTORY_EN	0	DFI-wire command-history scoreboard enable (sim only)
HIST_T_RCD / HIST_T_RP / HIST_T_RAS / HIST_T_RFC / HIST_T_WTR / HIST_T_RTW	0	Command-history timing constants
BL_WORDS	derived	Alias of BURST_WORDS
RD_MAX_OUTSTANDING	16	Read-aligner outstanding-read tracking depth
RD_EN_CYC	derived	rddata_en window width
CMD_DW	derived	Command FIFO payload width
WD_DW	derived	Write-data FIFO payload width {last,strb,data}
RD_DW	derived	Read-return FIFO payload width {last,resp,data}

10.10 Ports

10.10.1 Controller domain (ctl_clk)

Signal	Direction	Width	Description
ctl_clk	in	1	Controller clock
ctl_rstn	in	1	Controller reset (active low)
cmd_valid_i	in	1	Command stream valid (from scheduler)
cmd_ready_o	out	1	Command stream ready
cmd_data_i	in	[CMD_DW-1:0]	Packed {op,rank,bank,row,col,ap}
wd_valid_i	in	1	Write-data stream valid (from WR CAM)
wd_ready_o	out	1	Write-data stream ready
wd_data_i	in	[WD_DW-1:0]	Packed {last,strb,data} DFI-word granular
rd_valid_o	out	1	Read-return stream valid (to RD CAM)
rd_ready_i	in	1	Read-return stream ready
rd_data_o	out	[RD_DW-1:0]	Packed {last,resp,data}
init_start_i	in	1	Init-start request (controller side)
init_complete_o	out	1	Init-complete indication (controller side)

10.10.2 PHY DFI domain (dfi_clk)

Signal	Direction	Width	Description
dfi_clk	in	1	DFI / PHY clock
dfi_rstn	in	1	DFI reset (active low)

Signal	Direction	Width	Description
memtype_i	in	memtype_e	DDR2 / LPDDR2 selector
rd_phase_i	in	[PHW-1:0]	DFI phase carrying READ command
wr_phase_i	in	[PHW-1:0]	DFI phase carrying WRITE command
t_phy_wrlat_i	in	[7:0]	WR cmd → dfi_wrdata_en delay
t_rddata_en_i	in	[7:0]	RD cmd → dfi_rddata_en window delay
gear_i	in	[1:0]	Active gear: active_rate = 1 << gear_i
n_subcmd_i	in	[SUBW_MAX-1:0]	Active sub-column commands per DFI word
sub_col_stride_i	in	[COL_WIDTH-1:0]	Active device-word column stride
sub_phase_stride_i	in	[PHW-1:0]	Active DFI-phase stride between sub-bursts
dfi_address_o	out	[DFI_ADDR_BUS_W-1:0]	DFI address bus
dfi_bank_o	out	[DFI_BANK_BUS_W-1:0]	DFI bank bus
dfi_cas_n_o	out	[DFI_CTRL_BUS_W-1:0]	DFI CAS# (active low)
dfi_ras_n_o	out	[DFI_CTRL_BUS_W-1:0]	DFI RAS# (active low)
dfi_we_n_o	out	[DFI_CTRL_BUS_W-1:0]	DFI WE# (active low)
dfi_cs_n_o	out	[DFI_CS_BUS_W-1:0]	DFI chip-select (active low)
dfi_odt_o	out	[DFI_CS_BUS_W-1:0]	DFI ODT
dfi_wrdata_o	out	[DFI_DATA_WIDTH-1:0]	DFI write data
dfi_wrdata_en_o	out	[DFI_EN_WIDTH-1:0]	DFI write-data enable

Signal	Direction	Width	Description
dfi_wrdata_mask_o	out	[DFI_STRB_WIDTH-1:0]	DFI write-data mask
dfi_rddata_en_o	out	[DFI_EN_WIDTH-1:0]	DFI read-data enable
dfi_rddata_i	in	[DFI_DATA_WIDTH-1:0]	DFI read data
dfi_rddata_valid_i	in	[DFI_VALID_WIDTH-1:0]	DFI read-data valid
cal_expect_i	in	1	Calibration capture arm
cal_data_o	out	[DFI_DATA_WIDTH-1:0]	Calibration captured data
cal_valid_o	out	1	Calibration capture valid
dfi_init_start_o	out	1	DFI init-start to PHY
dfi_init_complete_i	in	1	DFI init-complete from PHY

10.11 Open questions

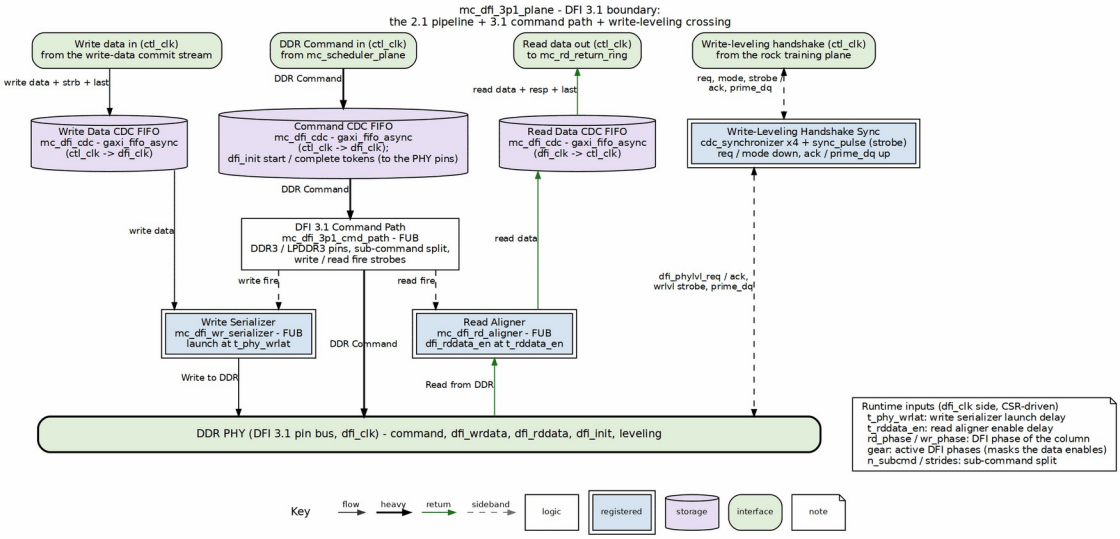
- The `fire_rank_o` output of `mc_dfi_cmd_path` is currently unused inside `mc_dfi_2p1_plane` (`macro/mc_dfi_2p1_plane.sv:375` ties it off). Open question: is this a reserved hook for per-rank ODT or training, or should it be removed?
- `db_wr_dbi_en_i`/`wd_dbi_i` and `rd_dbi_en_i`/`dfi_rddata_dbi_i` are tied off in the 2.1 plane (`macro/mc_dfi_2p1_plane.sv:336-337`, `macro/mc_dfi_2p1_plane.sv:367-369`), matching the DFI 2.1 no-DBI contract. The serializer/aligner `HAS_DBI` parameters default to 0 and are not exposed by the plane.
- Re-leveling (re-runs of `init_start`/`init_complete`) is marked future work in the CDC comments because the event-token crossing is monotonic (`fub/mc_dfi_cdc.sv:22`).

11 Chapter 8 — mc_dfi_3p1_plane

Module: macro/mc_dfi_3p1_plane.sv
Family boundary: DFI 3.1 controller/PHY seam
Status: extracted from scoria_dfi_plane during the Phase 2 common-plane reorg; consumed today by scoria_core (research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_core.sv:866).

11.1 Purpose and scope

`mc_dfi_3p1_plane` holds the single `ctl_clk`/`dfi_clk` crossing in the controller and presents the controller-clock command/write-data/read-data streams on one side and the DFI 3.1 pin bus on the other (`macro/mc_dfi_3p1_plane.sv:8-12`). It is a constant-latency conduit: all JEDEC spacing is enforced upstream in the scheduler, and this plane must not stall the in-order command stream (`fub/mc_dfi_3p1_cmd_path.sv:160-184`).



mc_dfi_3p1_plane internal block diagram

11.2 What is inherited unchanged from the 2.1 pipeline

The internal datapath unit is one DFI word; one FIFO word equals one DFI cycle, so the crossing is bubble-free (`macro/mc_dfi_3p1_plane.sv:19-20`). The 2.1 and 3.1 planes share these child instances and behaviours:

- **`mc_dfi_cdc`** — the single async-FIFO crossing for command, write data, read data, and init start/complete (`macro/mc_dfi_3p1_plane.sv:254`).
- **`mc_dfi_wr_serializer`** — delays write data by `t_phy_wrlat_i` and drives `dfi_wrdata_o`/`dfi_wrdata_en_o`/`dfi_wrdata_mask_o` (`macro/mc_dfi_3p1_plane.sv:350`).
- **`mc_dfi_rd_aligner`** — delays `dfi_rddata_en_o` by `t_rddata_en_i` and captures read data into the return FIFO (`macro/mc_dfi_3p1_plane.sv:373`).
- **Active-gear phase mask** — `gear_i` selects the active DFI rate; the unmasked `dfi_wrdata_en`/`dfi_rddata_en` from the serializer/aligner are gated by `w_phase_active` before the pins. At `gear_i == log2(DFI_RATE)` the mask is all-ones (`macro/mc_dfi_3p1_plane.sv:221-238`).

11.3 What the 3.1 delta adds

The generation delta is concentrated in the command path and the write-leveling surface.

11.3.1 Command path: `mc_dfi_3p1_cmd_path`

The 3.1 plane instantiates `fub/mc_dfi_3p1_cmd_path.sv` (`macro/mc_dfi_3p1_plane.sv:300`) in place of the 2.1 `mc_dfi_cmd_path`. The 3.1 path:

- Drops the optional 2.1 mode-register-read bit; the command word is `{ap, col, row, bank, rank, op}` (`fub/mc_dfi_3p1_cmd_path.sv:99-101`, `fub/mc_dfi_3p1_cmd_path.sv:158`).
- Formats commands through the silicon-proven `scoria_dfi_cmd_formatter` (`fub/mc_dfi_3p1_cmd_path.sv:255`).
- Supports sub-DFI-word burst packing via `N_SUBCMD`, `SUB_COL_STRIDE`, and `SUB_PHASE_STRIDE`: when `N_SUBCMD > 1`, one scheduled column command fans out into `N_SUBCMD` sub-commands placed on distinct DFI phases of the same cycle, and the per-sub buses merge by AND for control/CS and OR for address/bank/ODT (`fub/mc_dfi_3p1_cmd_path.sv:199-215`, `fub/mc_dfi_3p1_cmd_path.sv:291-315`).
- Uses the `scoria_cmd_history_checker` when `CMD_HISTORY_EN` is set (`fub/mc_dfi_3p1_cmd_path.sv:333-352`).

Clause-level DFI 3.1 pin semantics and the per-PHY integration contract are owned by the scoria book (`../../../research-ip/scoria-ddr3-lpddr3/docs/scoria_mas/ch03_interfaces/02_dfi_v31_pins.md`); this chapter does not restate them.

11.3.2 Write-leveling CDC

The 3.1 plane adds the DDR3 write-leveling handshake crossing. Because the plane owns the only `ctl_clk`/`dfi_clk` boundary, routing these signals through it avoids an unsynchronized crossing (`macro/mc_dfi_3p1_plane.sv:184-189`). Slow levels use `cdc_synchronizer` flop chains; the one-cycle `wl_strobe_i` pulse uses `sync_pulse` so the pulse count survives arbitrary clock ratios (`macro/mc_dfi_3p1_plane.sv:192-195`, `macro/mc_dfi_3p1_plane.sv:425-470`). Active-low `_n` signals are inverted to active-high for synchronization and inverted again at the output to avoid spurious assertions out of reset (`macro/mc_dfi_3p1_plane.sv:409-422`).

The scoria book states that this PHY family does not implement the DFI leveling handshake, so on the Genesys 2 build these pins are unused and write leveling is the PHY's own (`../../../research-ip/scoria-ddr3-lpddr3/docs/scoria_mas/ch03_interfaces/02_dfi_v31_pins.md:123`). The protocol state machine is documented in the scoria MAS (`../../../research-ip/scoria-ddr3-lpddr3/docs/scoria_mas/ch02_blocks/20_wrlvl_ifc.md`).

11.3.3 Calibration sideband removed

The 2.1 plane exposes `cal_expect_i`, `cal_data_o`, and `cal_valid_o` on the `dfi_clk` domain (`macro/mc_dfi_2p1_plane.sv:173-176`). The 3.1 plane removes these ports and ties `HAS_CAL_CAPTURE` to 0 inside `mc_dfi_rd_aligner` (`macro/mc_dfi_3p1_plane.sv:380`).

11.4 Parameters

Table 8.1: `mc_dfi_3p1_plane` parameters

Parameter	Default	Description
NUM_RANKS	1	DRAM ranks.
NUM_CS	NUM_RANKS	Chip selects.
NUM_BANKS	8	DRAM banks.
ROW_WIDTH	14	Row address width.
COL_WIDTH	10	Column address width.
DFI_RATE	2	Compile-time max DFI phases per clock.
DRAM_BEAT_WIDTH	64	DRAM beat width.
DFI_BEATS_PER_BURST	8	DRAM beats per burst (BL_PUMICE).
N_SUBCMD	1	Compile-time max sub-commands packed into one DFI word.
SUB_COL_STRIDE	1	Compile-time max column stride between sub-bursts.
SUB_PHASE_STRIDE	(N_SUBCMD>1)? (DFI_RATE/N_SUBCMD):1	Compile-time max DFI-phase stride between packed sub-bursts.
SUBW_MAX	$clog2(N_SUBCMD+1)	Width holding active N_SUBCMD.
BURST_WORDS	(DFI_BEATS_PER_BURST>=DFI_RATE)? (DFI_BEATS_PER_BURST/DFI_RATE):1	DFI words per burst group; clamped to 1 in the sub-word regime.
CMD_FIFO_DEPTH	8	Command async FIFO depth.
WD_FIFO_DEPTH	16	Write-data async FIFO depth.
RD_FIFO_DEPTH	32	Read-data async FIFO depth. Required because

Parameter	Default	Description
		dfi_rddata_valid is fire-and-forget; depth 16 dropped beats under paced adjacent bursts (macro/mc_dfi_3p1_plane.sv:56-64).
N_FLOP_CROSS	2	Pointer synchronizer flop stages.
USE_JOHNSON	0	Async FIFO pointer encoding: 0 = Gray, 1 = Johnson.
CMD_HISTORY_EN	0	Enable DFI-wire command-history scoreboard (sim only).
HIST_T_RCD	0	History checker tRCD.
HIST_T_RP	0	History checker tRP.
HIST_T_RAS	0	History checker tRAS.
HIST_T_RFC	0	History checker tRFC.
HIST_T_WTR	0	History checker tWTR.
HIST_T_RTW	0	History checker tRTW.
BL_WORDS	BURST_WORDS	Alias exposed to the read aligner.
RD_MAX_OUTSTANDING	16	Read aligner outstanding-read tracking depth.
RD_EN_CYC	BL_WORDS	dfi_rddata_en window width. The default is wrong when DRAM_BEAT_WIDTH > DRAM_DEVICE_WIDTH; scoria overrides explicitly (macro/mc_dfi_3p1_plane.sv:104-112).

Derived geometry parameters (DFI_DATA_WIDTH, DFI_STRB_WIDTH, DFI_EN_WIDTH, DFI_VALID_WIDTH, DFI_ADDR_WIDTH, DFI_BANK_WIDTH, DFI_CTRL_WIDTH, DFI_CS_WIDTH, the *_BUS_W widths, RKW, BKW, PHW, and FIFO payload widths CMD_DW/WD_DW/RD_DW) are computed in macro/mc_dfi_3p1_plane.sv:71-116.

11.5 Ports

11.5.1 Controller domain (`ctl_clk`)

Table 8.2: Controller-domain ports

Signal	Direction	Width	Description
`ctl_clk`	in	1	Controller clock.
`ctl_rstn`	in	1	Controller reset, active-low.
`cmd_valid_i`	in	1	Command valid from scheduler.
`cmd_ready_o`	out	1	Command ready to scheduler.
`cmd_data_i`	in	`CMD_DW`	Packed command `{ap, col, row, bank, rank, op}`.
`wd_valid_i`	in	1	Write-data valid.
`wd_ready_o`	out	1	Write-data ready.
`wd_data_i`	in	`WD_DW`	Write data `{last, strb, data}`.
`rd_valid_o`	out	1	Read-data valid.
`rd_ready_i`	in	1	Read-data ready.
`rd_data_o`	out	`RD_DW`	Read data `{last, resp, data}`.
`init_start_i`	in	1	Init start level.
`init_complete_o`	out	1	Init complete sticky.

11.5.2 PHY DFI domain (`dfi_clk`)

Table 8.3: PHY DFI-domain ports

Signal	Direction	Width	Description
`dfi_clk`	in	1	DFI clock.
`dfi_rstn`	in	1	DFI reset, active-low.
`memtype_i`	in	`memtype_e`	DRAM type

Signal	Direction	Width	Description
			(DDR3/LPDDR3).
rd_phase_i	in	PHW	Read-command DFI phase.
wr_phase_i	in	PHW	Write-command DFI phase.
t_phy_wrlat_i	in	8	WR command to dfi_wrdata_en delay.
t_rddata_en_i	in	8	RD command to dfi_rddata_en delay.
gear_i	in	2	Active DFI gear ratio selector.
n_subcmd_i	in	SUBW_MAX	Active sub-commands per DFI word.
sub_col_stride_i	in	COL_WIDTH	Active column stride.
sub_phase_stride_i	in	PHW	Active phase stride.
dfi_address_o	out	DFI_ADDR_BUS_W	DFI address bus.
dfi_bank_o	out	DFI_BANK_BUS_W	DFI bank bus.
dfi_cas_n_o	out	DFI_CTRL_BUS_W	DFI CAS#.
dfi_ras_n_o	out	DFI_CTRL_BUS_W	DFI RAS#.
dfi_we_n_o	out	DFI_CTRL_BUS_W	DFI WE#.

Signal	Direction	Width	Description
dfi_cs_n_o	out	DFI_CS_BUS_W	DFI chip select.
dfi_odt_o	out	DFI_CS_BUS_W	DFI ODT.
dfi_wrdata_o	out	DFI_DATA_WIDTH	DFI write data.
dfi_wrdata_en_o	out	DFI_EN_WIDTH	DFI write data enable.
dfi_wrdata_mask_o	out	DFI_STRB_WIDTH	DFI write data mask.
dfi_rddata_en_o	out	DFI_EN_WIDTH	DFI read data enable.
dfi_rddata_i	in	DFI_DATA_WIDTH	DFI read data.
dfi_rddata_valid_i	in	DFI_VALID_WIDTH	DFI read data valid.
dfi_init_start_o	out	1	DFI init start sticky.
dfi_init_complete_i	in	1	DFI init complete level.

11.5.3 Write-leveling crossing ports

Table 8.4: Write-leveling crossing ports

Signal	Direction	Width	Description
wl_req_cs_n_i	in	NUM_CS	Leveling request from scheduler, active-low.
wl_wrlvl_cs_n_i	in	NUM_CS	WRLVL mode from scheduler, active-low.
wl_strobe_i	in	1	One-cycle strobe from scheduler.
wl_ack_cs_n_o	out	NUM_CS	PHY ack to scheduler, active-low.
wl_prime_dq_o	out	1	Prime DQ to

Signal	Direction	Width	Description
			scheduler.
dfi_phylvl_req_cs_n_o	out	NUM_CS	Leveled request to PHY, active-low.
dfi_phylvl_ack_cs_n_i	in	NUM_CS	PHY ack, active-low.
dfi_phy_wrlvl_cs_n_o	out	NUM_CS	Leveled WRLVL mode to PHY, active-low.
dfi_wrlvl_strobe_o	out	1	Leveled one-cycle strobe to PHY.
dfi_prime_dq_i	in	1	Prime DQ from PHY.

11.6 Family-boundary doctrine

DFI revision is a family boundary, not a tunable parameter: a given controller instance commits to one DFI generation at compile time. The rationale and lineage are in the architecture book (`../common_has/ch03_dfi_generations/01_dfi_planes.md`).

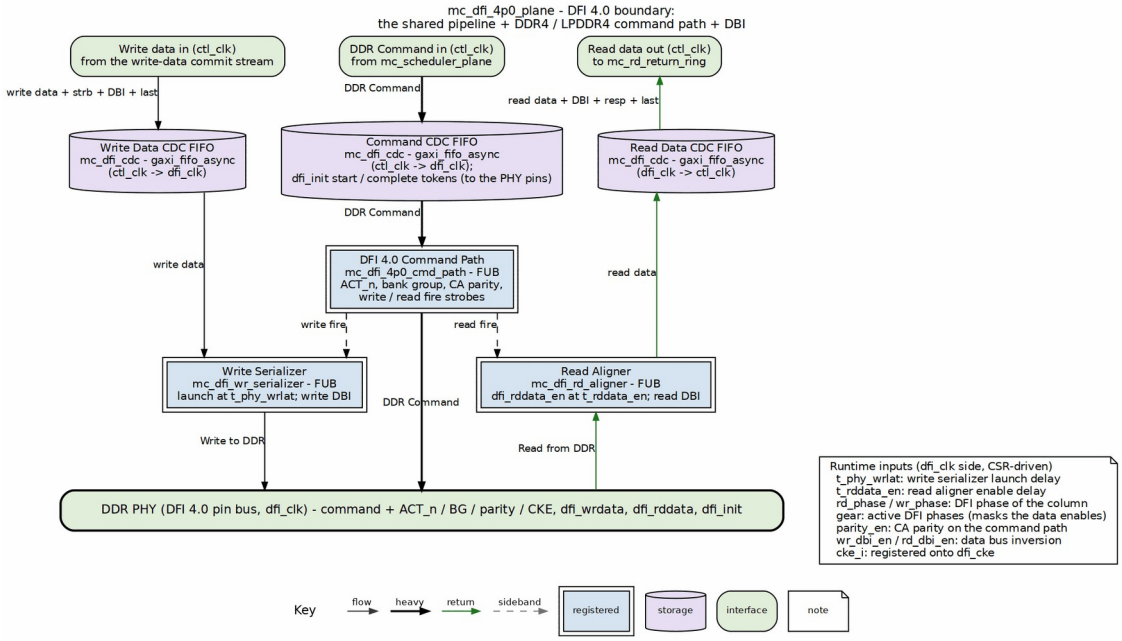
11.7 Open questions and verification notes

- `scoria_core.sv` at `research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_core.sv:866` instantiates `mc_dfi_3p1_plane` and wires every write-leveling port (`research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_core.sv:937-947`), yet a stale comment at `research-ip/scoria-ddr3-lpddr3/rtl/top/scoria_core.sv:234` claims the handshake does not go through the plane. The RTL wiring is the operative fact; the comment is inconsistent.
- The sibling scoria MAS chapter (`../../../research-ip/scoria-ddr3-lpddr3/docs/scoria_mas/ch02_blocks/03_dfi_plane.md`) still documents the removed `scoria_dfi_plane`; no `scoria_dfi_plane.sv` or `scoria_dfi_cmd_path.sv` remains in the scoria RTL tree (`Glob` returned no matches). That chapter is therefore out of date with respect to current RTL.

---

12 mc_dfi_4p0_plane — DFI 4.0 Generation Delta

mc_dfi_4p0_plane is the common DFI 4.0 boundary block extracted from andesite_dfi_plane during the Phase 2 common-plane reorganization (mc_dfi_4p0_plane.sv:20). It holds the single controller-clock to DFI-clock crossing and the dfi_clk-domain datapath that turns the family streams into the DFI 4.0 pin surface.



mc_dfi_4p0_plane internal block diagram

DFI 4.0 = shared pipeline + DDR4/LPDDR4 command surface

12.1 Inherited shared pipeline

The plane instantiates four shared FUBs (`mc_dfi_4p0_plane.sv:14-18`): `mc_dfi_cdc` for the async command/write/read/init-token crossings (`mc_dfi_4p0_plane.sv:220-259`), `mc_dfi_4p0_cmd_path` for the widened scheduler word to DFI 4.0 pins (`mc_dfi_4p0_plane.sv:266-297`), `mc_dfi_wr_serializer` with `HAS_DBI=1` (`mc_dfi_4p0_plane.sv:302-321`), and `mc_dfi_rd_aligner` with `HAS_DBI=1` (`mc_dfi_4p0_plane.sv:326-354`). Training pins live on rock-specific `mc_training_plane` wrappers, not here (`mc_dfi_4p0_plane.sv:25`).

12.2 What the 4.0 delta adds

12.2.1 Command path (`mc_dfi_4p0_cmd_path`)

The command path accepts the widened word `{ap, col, row, bg, bank, rank, op}` (`mc_dfi_4p0_cmd_path.sv:91-99`); the `bg` field is the structural delta over the 3.1 word (`mc_dfi_4p0_cmd_path.sv:22-23`). It presents each accepted command to the rock-local P1 single-shot formatter `andesite_dfi_cmd_formatter` for one cycle and exposes the registered DFI pin vector plus fire strobes (`mc_dfi_4p0_cmd_path.sv:8-13`). Structural readiness is the only backpressure (`mc_dfi_4p0_cmd_path.sv:104-109`), per the carried scoria `TASK-007` contract (`mc_dfi_4p0_cmd_path.sv:107`). The `CMD_HISTORY_EN` parameter is audit-only and not instantiated in v1 (`mc_dfi_4p0_cmd_path.sv:41-45`). Address routing: `OP_MRS`/`OP_ZQCL` payload rides `ROW`; `OP_RD`/`OP_RDA`/`OP_WR`/`OP_WRA` merge `ap` into JEDEC A10; `OP_PREA` forces A10 high and `OP_PRE` clears it (`mc_dfi_4p0_cmd_path.sv:145-152`).

ACT_n. `dfi_act_n_o` is driven by `andesite_dfi_cmd_formatter` together with `dfi_ras_n_o`, `dfi_cas_n_o`, and `dfi_we_n_o` (`mc_dfi_4p0_cmd_path.sv:178-181`), forming the DFI 4.0 activate surface (andesite HAS §3.1 Control Interface).

Bank groups. `dfi_bg_o` is driven from the unpacked `w_bg` field (`mc_dfi_4p0_cmd_path.sv:95`, 142), giving the DFI 4.0 bank-group address (andesite HAS §3.1 Control Interface).

CA parity. `parity_en_i` feeds the formatter to generate `dfi_parity_in_o` (`mc_dfi_4p0_cmd_path.sv:60`, 176, 186; andesite HAS §3.5.7 CRC and CA Parity). The hw/fw split of any parity-error counting machinery is an open question inherited from the andesite HAS §TBC(TASK-005).

Gear-down. `gear_i[1:0]` selects the active DFI rate as `1 << gear_i`; the phase-active mask is applied to `dfi_wrdata_en_o` and `dfi_rddata_en_o` (`mc_dfi_4p0_plane.sv:182-195`). The andesite HAS states that the init sequencer drives gear-down entry after init (andesite HAS §3.13 Geardown Mode); that sequencing is not visible in this common block's RTL §TBC(TASK-005).

DBI. Write DBI travels with the CDC word `{last,strb,dbi,data}` and is driven through `dfi_wrdata_mask_o` (`mc_dfi_4p0_plane.sv:75`, 96, 148, 302-321). Read DBI enters on `dfi_rddata_dbi_i` and exits on `rd_dbi_o` (`mc_dfi_4p0_plane.sv:103`, 155, 326-354); both child instances set `HAS_DBI=1` (`mc_dfi_4p0_plane.sv:305`, 333). This matches the andesite HAS correction that DFI 4.0 has no `dfi_wrdata_dbi` pin (andesite HAS §3.2.1 Write Data Mask/Write DBI, §3.3.1 Read DBI).

Alert_n. `dfi_alert_n*` handling is not in this plane; the andesite HAS places it in a boundary `alert_n` submodule surfaced through `dfi_error` (andesite HAS §3.5.7 CRC and CA Parity), while the common plane leaves alerts to the rock training wrapper (`mc_dfi_4p0_plane.sv:25`) §TBC(TASK-005).

12.3 What the plane leaves unimplemented

Omission	Rationale	Citation
DDR5-oriented 4.x features	No DDR5 target	andesite HAS §2.2.1, §2.2.2
Frequency ratios beyond inherited 1:4	No board PHY to consume them	andesite HAS §4.9
`dfi_lp_*` request consumption	Dormant-pair condition; requests stay presented	andesite HAS §3.7
Training pins	Live on `mc_training_plane` rock wrappers	`mc_dfi_4p0_plane.sv:25`; andesite HAS §3.6

`dfi_cke_o` is registered from `cke_i`, not produced by the command-path formatter (`mc_dfi_4p0_plane.sv:125`, `362-373`). At `NUM_RANKS=1`, `dfi_wrdata_cs_o` and `dfi_rddata_cs_o` are driven zero, matching the scoria v3.1 precedent (`mc_dfi_4p0_plane.sv:30-31`, `359-360`).

12.4 Parameters

12.4.1 mc_dfi_4p0_plane

Parameter	Default	Description
NUM_RANKS	1	Ranks
NUM_BANKS	8	Banks per group
NUM_BG	4	Bank groups
ROW_WIDTH	14	Row width
COL_WIDTH	10	Column width
ADDR_WIDTH	18	DFI address width
DFI_RATE	2	DFI:DRAM clock ratio minus one (1:4 default)
DRAM_BEAT_WIDTH	64	DRAM beat width
DFI_DATA_WIDTH	DRAM_BEAT_WIDTH * DFI_RATE	DFI data width
CMD_FIFO_DEPTH	8	Command FIFO depth
WD_FIFO_DEPTH	16	Write-data FIFO depth
RD_FIFO_DEPTH	32	Read-data FIFO depth
RD_MAX_OUTSTANDING	16	Read-aligner outstanding depth
RD_EN_CYC	4	dfi_rddata_en window (DFI cycles)
BL_WORDS	RD_EN_CYC	DFI words per read
N_FLOP_CROSS	2	CDC synchronizer flops
USE_JOHNSON	0	0 = Gray, 1 = Johnson
DFI_STRB_WIDTH	DFI_DATA_WIDTH / 8	Byte/mask/DBI width
DFI_EN_WIDTH	DFI_RATE	dfi_*data_en width
DFI_VALID_WIDTH	DFI_RATE	dfi_rddata_valid width
BANK_WIDTH	$clog2(NUM_BANKS)	Bank address width
BG_WIDTH	2	Bank-group width
RKW	(NUM_RANKS > 1) ? $clog2(NUM_RANKS) : 1	Rank index width
BKW	$clog2(NUM_BANKS)	Bank index in command

Parameter	Default	Description
		word
BGW	(NUM_BG > 1) ? $clog2(NUM_BG) : 1	Bank-group index in command word
PHW	(DFI_RATE > 1) ? $clog2(DFI_RATE) : 1	Phase index width
CMD_DW	$bits(dram_op_e) + RKW + BKW + BGW + ROW_WIDTH + COL_WIDTH + 1	Command word {ap,col,row,bg,bank,rank,op}
WD_DW	1 + DFI_STRB_WIDTH + DFI_STRB_WIDTH + DFI_DATA_WIDTH	Write-data CDC payload {last,strb,dbi,data}
RD_DW	1 + 2 + DFI_STRB_WIDTH + DFI_DATA_WIDTH	Read-data CDC payload {last,resp,dbi,data}

12.5 Ports

12.5.1 mc_dfi_4p0_plane — controller domain (ctl_clk)

Port	Direction	Width	Description
ctl_clk	input	1	Controller clock
ctl_rstn	input	1	Controller reset
cmd_valid_i	input	1	Command valid
cmd_ready_o	output	1	Command ready
cmd_data_i	input	[CMD_DW-1:0]	Command word
wd_valid_i	input	1	Write-data valid
wd_ready_o	output	1	Write-data ready
wd_data_i	input	[DFI_DATA_WIDTH-1:0]	Write data
wd_strb_i	input	[DFI_STRB_WIDTH-1:0]	Write byte strobe
wd_dbi_i	input	[DFI_STRB_WIDTH-1:0]	Write DBI
wd_last_i	input	1	Last word of burst
rd_valid_o	output	1	Read-data valid
rd_ready_i	input	1	Read-data ready
rd_data_o	output	[DFI_DATA_WIDTH-1:0]	Read data
rd_dbi_o	output	[DFI_STRB_WIDTH-1:0]	Read DBI
rd_resp_o	output	[1:0]	Read response
rd_last_o	output	1	Last word of read return
init_busy_i	input	1	Macro init busy; crossed to dfi_init_start_o
init_done_i	input	1	Macro init done (unused)
rd_dbi_en_i	input	1	Read DBI enable
wr_dbi_en_i	input	1	Write DBI enable

Port	Direction	Width	Description
memtype_i	input	memtype_e	Memory type
parity_en_i	input	1	CA parity enable
gear_i	input	[1:0]	Gear-down selector
rd_phase_i	input	[PHW-1:0]	Read phase (unused)
wr_phase_i	input	[PHW-1:0]	Write phase (unused)
t_phy_wrlat_i	input	[7:0]	WR cmd to dfi_wrdata_en
t_rddata_en_i	input	[7:0]	RD cmd to dfi_rddata_en
cke_i	input	1	CKE from P1 init sequencer

12.5.2 mc_dfi_4p0_plane — PHY DFI domain (dfi_clk)

Port	Direction	Width	Description
dfi_clk	input	1	DFI clock
dfi_rstn	input	1	DFI reset
dfi_address_o	output	[ADDR_WIDTH-1:0]	Address
dfi_bank_o	output	[BANK_WIDTH-1:0]	Bank
dfi_bg_o	output	[BG_WIDTH-1:0]	Bank group
dfi_act_n_o	output	1	Activate command
dfi_ras_n_o	output	1	RAS_n
dfi_cas_n_o	output	1	CAS_n
dfi_we_n_o	output	1	WE_n
dfi_cs_o	output	[NUM_RANKS-1:0]	Chip select
dfi_cke_o	output	1	CKE, registered from cke_i
dfi_parity_in_o	output	1	CA parity
dfi_wrdata_o	output	[DFI_DATA_WIDTH-1:0]	Write data

Port	Direction	Width	Description
dfi_wrdata_en_o	output	[DFI_EN_WIDTH-1:0]	Write data enable, phase-masked
dfi_wrdata_mask_o	output	[DFI_STRB_WIDTH-1:0]	Write data mask / write DBI
dfi_wrdata_cs_o	output	[NUM_RANKS-1:0]	Write data chip select (zero at NUM_RANKS=1)
dfi_rddata_en_o	output	[DFI_EN_WIDTH-1:0]	Read data enable, phase-masked
dfi_rddata_i	input	[DFI_DATA_WIDTH-1:0]	Read data
dfi_rddata_valid_i	input	[DFI_VALID_WIDTH-1:0]	Read data valid
dfi_rddata_dbi_i	input	[DFI_STRB_WIDTH-1:0]	Read DBI
dfi_rddata_cs_o	output	[NUM_RANKS-1:0]	Read data chip select (zero at NUM_RANKS=1)
dfi_init_start_o	output	1	Init start, CDC token of init_busy_i
dfi_init_complete_i	input	1	Init complete

12.5.3 mc_dfi_4p0_cmd_path — dfi_clk domain

Port	Direction	Width	Description
dfi_clk	input	1	DFI clock
dfi_rstn	input	1	DFI reset
memtype_i	input	memtype_e	Memory type
parity_en_i	input	1	CA parity enable
cmd_valid_i	input	1	Command valid
cmd_ready_o	output	1	Command ready

Port	Direction	Width	Description
cmd_data_i	input	[CMD_DW-1:0]	Command word
rd_op_ready_i	input	1	Read-op ready from aligner
wr_op_ready_i	input	1	Write-op ready (staged token)
wr_fire_o	output	1	Write fired, registered
rd_fire_o	output	1	Read fired, registered
fire_rank_o	output	[RKW-1:0]	Rank of fired command
wr_accept_o	output	1	Pop staged write-data token
dfi_address_o	output	[ADDR_WIDTH-1:0]	Address
dfi_bank_o	output	[BANK_WIDTH-1:0]	Bank
dfi_bg_o	output	[BG_WIDTH-1:0]	Bank group
dfi_act_n_o	output	1	Activate command
dfi_ras_n_o	output	1	RAS_n
dfi_cas_n_o	output	1	CAS_n
dfi_we_n_o	output	1	WE_n
dfi_cs_o	output	[NUM_RANKS-1:0]	Chip select
dfi_cke_o	output	1	Placeholder CKE from formatter
dfi_parity_in_o	output	1	CA parity

12.6 Family boundary rule

The rule that assigns each controller generation to its DFI revision, and the doctrine for what belongs at the controller/PHY seam, is owned by the family architecture specification: see common_has chapter 3. The lineage summary lives at `../../03_dfi_boundary_lineage.md`, and the andesite HAS clause-level DFI 4.0 surface is at `../../../research-ip/andesite-ddr4-lpddr4/docs/andesite_has/ch04_interfaces/01_dfi_v40.md`.

12.7 Verification record

As of today, andesite's top-level RTL instantiates `mc_dfi_4p0_plane` directly (`research-ip/andesite-ddr4-lpddr4/rtl/top/andesite_core.sv:882`) and the andesite filelists pull in `common-ip/rtl/filelists/macro/mc_dfi_4p0_plane.f` (`research-ip/andesite-ddr4-lpddr4/rtl/filelists/andesite_all.f:19`, `andesite_core.f:46`). The rock-local `andesite_dfi_plane` is no longer the top boundary; only its P1 formatter `andesite_dfi_cmd_formatter` remains inside `mc_dfi_4p0_cmd_path`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 mc_init_plane — DRAM Initialization Plane

> Filed as MAS chapter 10 (appended) to keep chapters 2–9's paths stable for the ~50 inbound doc links; in command-flow order the init plane sits between the storage plane and the scheduler plane — see the family HAS plane stack, which is the binding statement of the architecture.

Extracted from `mc_scheduler_plane` on 2026-10-15 (spec: `docs/superpowers/specs/2026-10-15-mc-init-plane-design.md`). Both xDDR2 rocks — granite (product) and pumice (research) — instantiate this plane identically, so init work on one translates directly to the other. Scoria's wrapper continues to use the scheduler plane's legacy internal-init mode with its own rock-side sequencer; andesite is untouched.

13.1 Responsibility

Own the DRAM bring-up concern end to end:

1. **JEDEC initialization sequencing** (`mc_xddr2_init_sequencer`) — the full DDR2 chain and the LPDDR2 MRW chain, issued to the DRAM (not shadowed), with CSR-backed inter-command waits.
2. **Mode-register shadow + live decode** (`mc_xddr2_mode_register`) — the per-rank MR image, written by the sequencer during bring-up, decoded to CL/CWL/BL for the rest of the controller.
3. **DFI PHY init handshake** — `dfi_init_start_o` / `dfi_init_complete_i` crossed to the DFI plane; the FSM parks in `S_DFI_INIT` until the PHY reports done.
4. **Init command injection** — the plane's `init_cmd_*` outputs feed the scheduler plane's external-init port; the pick core forwards them to the command formatter while `!init_done`, so init commands reach the DRAM unarbitrated.
5. **Bring-up telemetry** — `init_busy_o` and `init_state_o[4:0]`.

Explicitly not owned: scheduling, refresh, the CSR/APB hot-update path for mode registers (future; `mr_seq_we` remains the only writer), and any training/search logic (the training plane's job).

13.2 Parameters and ports

Parameter	Default	Meaning
NUM_RANKS	1	rank count (mode-register shadow banks)
NUM_BANKS	8	bank count (MR-index width on the MRS bank port)
ROW_WIDTH	14	row/MR-data width (BKW = $clog2(NUM_BANKS))

Port	Dir	Width	Meaning
mc_clk, mc_rst_n	in	1	controller clock / active-low reset
memtype_i	in	memtype_e	DDR2 vs LPDDR2 chain select (sampled at the S_DFI_INIT exit and on restart)
t_init_wait_i, t_dll_wait_i	in	16	CKE/tINIT settle; DLL lock (tZQINIT on the LPDDR2 chain)
t_mrd_wait_i, t_rp_wait_i, t_rfc_wait_i	in	8	tMRD / tRP / tRFC post-command waits
mr0_i … mr3_i	in	16	DDR2 MR base values (CSR MR0..MR3.VAL)
lp_mr1_op_i, lp_mr2_op_i, lp_mr3_op_i	in	8	LPDDR2 MRW OP data (CSR LP_MR1/2/3_OP, JEDEC-default resets

Port	Dir	Width	Meaning
			0x23/0x01/0x02)
`init_restart_i`	in	1	rising-edge re-runs the chain with current CSR values (`CTRL.init_force_restart`)
`dfi_init_start_o` / `dfi_init_complete_i`	out/in	1	PHY init handshake (plane drives the DFI directly)
`init_cmd_valid_o`, `init_cmd_op_o`, `init_cmd_bank_o`, `init_cmd_row_o`	out	1 / 5 / BKW / ROW_WIDTH	command injection into the scheduler plane external-init port
`init_busy_o`, `init_done_o`	out	1	FSM status
`init_state_o`	out	5	raw FSM state mirror — telemetry, not a stable ABI
`cl_o`, `cwl_o`, `bl_o`	out	4	live decode of the MR shadow (CL/CWL/BL)

13.3 The two JEDEC sequences

DDR2 (waits in MC cycles; mirrors LiteDRAM's `get_ddr2_phy_init_sequence`, proven on the Nexys A7): PREA → EMR2 → EMR3 → EMR1 → MR0+DLL_RESET → PREA → REF → REF → MR0 (DLL reset cleared) → EMR+OCD-default → EMR+OCD-exit → done. The MR shadow is written on EMR2/3/1, both MR0 loads, and the OCD-exit EMR; the transient OCD-default load writes no shadow. MR0 is the CSR base `mr0_i`; the first load ORs in the DLL-reset bit (0x100), the OCD-default load ORs A[9:7]=111 (0x380).

LPDDR2 (JESD209-2F §3.4/§3.5): MRW(MR63) Reset → MRW(MR10) ZQ Init → MRW(MR1) BL/nWR → MRW(MR2) RL/WL → MRW(MR3) DS → done. The MR index (MA) and data (OP) pack into the row request as `{MA[5:0], OP[7:0]}` (a 3-bit bank port cannot reach MR10/63); the command formatter unpacks it for the CA MRW word. Only MR1/2/3 update the shadow (as `{8'd0, OP}`); MR63/MR10 issue but are not shadowed. The OP data is CSR-backed (`lp_mr*_op_i`) so software can retune at bring-up — and sweep the values to defeat a scrambled CA/A-lane board mapping, the same escape hatch the DDR2 `MRx.VAL` registers provide.

`init_state_o` encoding (raw FSM enum): S_RESET=0, S_DFI_INIT=1, S_PREA1=2, S_EMR3=3, S_EMR2=4, S_EMR1=5, S_MR0_DLL=6, S_PREA2=7, S_REF1=8, S_REF2=9, S_MR0=10, S_OCD_DEF=11, S_OCD_EXIT=12, S_WAIT=13, S_DONE=14, S_L_RESET=15, S_L_ZQ=16, S_L_MR1=17, S_L_MR2=18, S_L_MR3=19.

13.4 Configuration surface

The plane consumes no parameters beyond geometry; every timing and MR value is a CSR port. Both rocks expose:

CSR	Offset	Reset	Notes
`INIT_TIMING0`	0x058	t_init 512 / t_dll 256	MC cycles — **these values are MC-frequency-dependent; re-derive them when the controller clock changes**
`INIT_TIMING1`	0x05C	t_mrd 8 / t_rp 8 / t_rfc 16	same frequency caution
`MR0..MR3.VAL`	0x020–0x02C	MR0 = 0x433	DDR2 base values
`LP_MR1_OP`	0x180	0x23	LPDDR2 MR1 OP (nWR/WC/BT/BL)
`LP_MR2_OP`	0x184	0x01	LPDDR2 MR2 OP (RL/WL)
`LP_MR3_OP`	0x188	0x02	LPDDR2 MR3 OP (drive strength)
`CTRL.init_force_restart`	0x000	0	rising edge re-runs the chain

13.5 Verification

- `common-ip/dv/tests/fub/test_mc_xddr2_init_sequencer.py` — directed: both chains (order/shadows/waits), LP OP sweep, restart level/memtype behavior, `dfi_init_complete` never/glitch negatives, state telemetry.
- `common-ip/dv/tests/macro/test_mc_init_plane.py` — both chains at the plane boundary + CL/BL decode.
- Integration: granite `test_granite_core` / `test_granite_top_geared`, pumice `test_pumice_top.py` (incl. `lp_mr_op_csr_sweep_lpddr2`, which sweeps the LP CSRs and verifies the values in the decoded CA MRW stream via `init_force_restart`).